

强化学习面试课程 Roadmap

这套笔记面向 AI（Artificial Intelligence，人工智能）算法工程师的强化学习入门与面试准备。目标不是把强化学习研究方向穷尽，而是系统掌握面试中最常被考察的概念、公式、算法区别、训练技巧和项目表达方式。

全局符号和缩写说明

阅读约定：每个章节还会在开头重复列出本章会用到的核心符号。一般来说，大写字母表示随机变量、函数或集合，小写字母表示一次采样到的具体取值；例如 S_t 是时刻 t 的状态随机变量， s 是某个具体状态。

注意符号会按上下文复用：例如 A 可以表示 action space（动作空间），而 $A_{\pi}(s,a)$ 可以表示 advantage function（优势函数）； $P(s'|s,a)$ 是 transition probability， $P(i)$ 则是采样概率。

本套笔记统一使用大写 $V_{\pi}(s)$ 和 $Q_{\pi}(s,a)$ 表示价值函数；有些教材会写成小写 $v_{\pi}(s)$ 、 $q_{\pi}(s,a)$ ，含义相同，只是记号风格不同。

符号/缩写	English	中文	含义
AI	Artificial Intelligence	人工智能	本课程面向 AI 算法工程师的面试准备。
RL	Reinforcement Learning	强化学习	agent 通过交互学习最大化长期 reward 的范式。
MDP	Markov Decision Process	马尔可夫决策过程	RL 最常见的数学建模。
DP	Dynamic Programming	动态规划	已知环境模型时用 Bellman backup 求解 MDP 的方法。
MC	Monte Carlo	蒙特卡洛	用完整 episode 的真实 return 估计价值。
TD	Temporal Difference	时序差分	用一步 reward 加下一状态估计值做 bootstrap 更新。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	on-policy TD control 算法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	最基础的 Monte Carlo policy gradient 算法。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似动作价值函数 $Q(s,a)$ 。
TRPO	Trust Region Policy Optimization	信赖域策略优化	通过 KL 约束限制策略更新幅度。

符号/缩写	English	中文	含义
PPO	Proximal Policy Optimization	近端策略优化	用 clipping objective 稳定策略更新。
SAC	Soft Actor-Critic	软 Actor-Critic	最大熵 actor-critic 算法。
DDPG	Deep Deterministic Policy Gradient	深度确定性策略梯度	连续动作空间的 off-policy actor-critic 算法。
TD3	Twin Delayed DDPG	双延迟 DDPG	改进 DDPG 稳定性的连续控制算法。
A2C / A3C	Advantage Actor-Critic / Asynchronous Advantage Actor-Critic	优势 Actor-Critic / 异步优势 Actor-Critic	actor-critic 的同步和异步版本。
GAE	Generalized Advantage Estimation	广义优势估计	用 λ 平衡 advantage 估计的 bias 和 variance。
UCB	Upper Confidence Bound	置信上界	bandit 中基于不确定性 bonus 的探索方法。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好信号优化模型行为。
RM	Reward Model	奖励模型	RLHF 中根据偏好数据学习出的奖励模型。
S / S_t	State / State at time t	状态空间 / 时刻 t 的状态	S 常表示状态空间, S_t 表示状态随机变量。
A / A_t	Action / Action at time t	动作空间 / 时刻 t 的动作	A 常表示动作空间, A_t 表示动作随机变量。
R / R_{t+l}	Reward	奖励	R_{t+l} 表示执行 A_t 后收到的即时奖励。
G_t	Return	回报 / 累计折扣奖励	G_t 表示从时刻 t 开始的累计收益; G 是常用记号, 不建议理解成固定英文缩写。
$P(s' s, a)$	Transition Probability	状态转移概率	在状态 s 执行动作 a 后转移到 s' 的概率。
$V_\pi(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的期望 return。
$Q_\pi(s, a)$	Action-value Function	动作价值函数	策略 π 下, 在 s 先执行 a 的期望 return; Q 可理解为 action quality。
$A_\pi(s, a)$	Advantage Function	优势函数	动作 a 相对状态平均价值的优势; 这里的 A 不是 action space。

符号/缩写	English	中文	含义
π	Policy	策略	从 state 到 action 或 action distribution 的映射。
γ	Discount Factor	折扣因子	控制未来 reward 的折扣强度。
θ / w	Parameters / Weights	参数 / 权重	神经网络或函数逼近器的可学习参数。
$J(\theta)$	Objective Function	优化目标函数	policy optimization 中要最大化的目标。
$L(\theta)$	Loss Function	损失函数	训练神经网络时要最小化的目标。
$H(\pi)$	Entropy	熵	衡量策略随机性，常用于鼓励探索。
KL	Kullback-Leibler Divergence	KL 散度	衡量两个概率分布差异。

学习目标

学完后你应该能做到：

- 清楚解释强化学习和监督学习的区别。
- 熟练写出 MDP、return、value function、Bellman equation。
- 区分 DP、Monte Carlo、TD、SARSA、Q-learning。
- 解释 DQN 为什么需要 replay buffer 和 target network。
- 推导 policy gradient 的核心公式。
- 讲清 Actor-Critic、GAE、PPO、SAC 的动机。
- 理解 offline RL、imitation learning、RLHF 的基础问题。
- 准备一套可以在面试中讲清楚的 RL 项目。

全课程主线

这套课按“问题建模 -> 精确求解 -> 采样估计 -> 函数逼近 -> 策略优化 -> 进阶约束”的顺序组织。每章不是孤立算法，而是在回答上一个阶段留下的问题：

章节	主要问题	逻辑位置
第 1 章	RL 问题如何写成 MDP、return、value 和 Bellman 方程	建立统一语言
第 2 章	如果环境模型已知，怎样用 DP 精确做 evaluation 和 improvement	Bellman 方程的精确求解原型
第 3 章	如果模型未知，怎样用采样做 prediction 和 control	从 DP 过渡到 model-free tabular RL

章节	主要问题	逻辑位置
第 4 章	学 Q 或 policy 时，数据从哪里来，如何平衡探索和利用	解释采样数据质量
第 5 章	表格法放不下真实状态空间时，怎样用函数逼近	进入 deep RL 的必要前提
第 6 章	Q-learning 加神经网络为什么会不稳，DQN 如何稳定它	value-based deep RL
第 7 章	不想通过 $\arg\max Q$ 选动作时，如何直接优化 policy	policy-based RL
第 8 章	如何同时利用 policy gradient 和 value learning	actor-critic 框架
第 9 章	policy 更新太大导致崩溃时，如何限制更新幅度	现代 on-policy 稳定训练
第 10 章	连续动作下无法枚举动作时，如何做 off-policy actor-critic	连续控制
第 11 章	交互、模型、离线数据、人类反馈带来哪些新约束	高级 RL 与 LLM 对齐接口
第 12 章	面试中如何横向比较、推公式、讲项目	收束成可表达知识体系

推荐学习节奏

建议周期：6 到 8 周。

建议投入：每周 8 到 10 小时。

每章建议按这个顺序学习：

1. 先读核心概念。
2. 手写关键公式。
3. 用自己的话讲一遍算法直觉。
4. 回答本章面试高频问题。
5. 至少实现或阅读一个最小代码版本。

推导阅读方式

RL 的公式不要直接背结论，建议按下面链路推：

1. 先写目标：agent 要最大化长期 return，而不是单步 reward。
2. 再拆递归：把 G_t 写成 $R_{t+1} + \gamma G_{t+1}$ ，自然得到 Bellman 思路。
3. 再区分期望和采样：Bellman 方程是期望形式，TD、SARSA、Q-learning 是用一次 transition 近似这个期望。
4. 再看函数逼近：DQN 把 tabular update 变成对 Bellman target 的监督回归。
5. 再看策略优化：policy gradient 用 log-derivative trick 处理“采样分布本身依赖参数”的问题。

6. 最后看稳定化：baseline、critic、GAE、target network、replay、PPO clipping、SAC entropy 都是在处理方差、偏差、分布漂移或探索。

面试里如果一时忘公式，可以从这条链路往回推：目标是什么，target 是什么，target 为什么可靠，更新过大或估计不准会出什么问题。

例子阅读方式

每章都补了“本章例子”或“典型例子”。读例子时不要只记场景名，要把它映射回四件事：

1. 状态是什么。
2. 动作是什么。
3. 奖励怎么定义。
4. 算法为什么适合这个场景。

例如 CartPole 不是“一个小游戏”，而是：

元素	例子
状态	小车位置、速度、杆角度、角速度。
动作	向左推或向右推。
奖励	杆保持不倒时每步 +1。
算法	离散动作、低维状态时可用 DQN 或 PPO。

面试时讲例子的目的不是展示你见过环境，而是证明你能把业务问题建模成 MDP，再解释为什么某个算法匹配这个 MDP。

章节目录

章节	文件	主题
第 1 章	01-rl-fundamentals-mdp-bellman	RL 基本设定、MDP、价值函数、Bellman 方程
第 2 章	02-dynamic-programming	Policy Evaluation、Policy Iteration、Value Iteration
第 3 章	03-monte-carlo-td-learning	Monte Carlo、TD、SARSA、Q-learning
第 4 章	04-exploration-bandit	探索与利用、Multi-Armed Bandit、UCB、Thompson Sampling
第 5 章	05-function-approximation-deadly-triad	函数逼近、稳定性、Deadly Triad
第 6 章	06-dqn-and-variants	DQN、Double DQN、Dueling DQN、Prioritized Replay
第 7 章	07-policy-gradient	REINFORCE、Policy Gradient、baseline、advantage
第 8 章	08-actor-critic-gae	Actor-Critic、A2C、A3C、GAE

章节	文件	主题
第 9 章	09-ppo-trpo	TRPO、PPO、KL 约束、clipping objective
第 10 章	10-continuous-control-ddpg-td3-sac	DDPG、TD3、SAC、连续动作控制
第 11 章	11-model-based-offline-imitation-rlhf	Model-based RL、Offline RL、Imitation Learning、RLHF
第 12 章	12-interview-cheatsheet-projects	面试速查、横向对比、项目准备

面试复习优先级

如果时间有限，优先掌握：

1. MDP、Bellman 方程、V/Q 函数。
2. MC vs TD, SARSA vs Q-learning。
3. on-policy vs off-policy。
4. DQN 的 replay buffer、target network、Double DQN。
5. Policy Gradient 推导，baseline 为什么不引入 bias。
6. Actor-Critic 和 GAE。
7. PPO clipping objective。
8. SAC 的 entropy regularization。
9. offline RL 为什么难。
0. RLHF 的 PPO/RM/Preference data 基本流程。

推荐资料

- Sutton & Barto, Reinforcement Learning: An Introduction
- David Silver Reinforcement Learning Course
- OpenAI Spinning Up
- CleanRL single-file implementations
- Stable-Baselines3 文档和示例

面试表达模板

学习每个算法时，都尝试用这个模板总结：

算法名：
解决的问题：
核心思想：
目标函数或更新公式：
on-policy/off-policy：
value-based/policy-based/actor-critic：
稳定训练技巧：
优点：
缺点：
常见面试追问：

第 1 章：强化学习基本设定、MDP（Markov Decision Process，马尔可夫决策过程）与 Bellman 方程

本章符号说明

本章第一次出现公式前，先把核心符号列清楚：

本章统一写大写 $V_\pi(s)$ 和 $Q_\pi(s,a)$ ；如果你在 Sutton & Barto 等教材里看到小写 $v_\pi(s)$ 、 $q_\pi(s,a)$ ，它们表示同类 state-value/action-value function。

符号	English	中文	含义
S / S_t	State / State at time t	状态空间 / 时刻 t 的状态	S 是状态空间， S_t 是时刻 t 的状态随机变量。
s	State realization	具体状态	小写 s 表示某一个具体 state，不是 value function。
A / A_t	Action / Action at time t	动作空间 / 时刻 t 的动作	A 是动作空间， A_t 是时刻 t 的动作随机变量。
a	Action realization	具体动作	小写 a 表示某一个具体 action，不是 action-value function。
R / R_{t+l}	Reward	奖励	R_{t+l} 是执行 A_t 后得到的即时奖励。
G_t	Return	回报 / 累计折扣奖励	G_t 表示从时刻 t 开始的长期累计收益； G 是常用记号，不建议理解成固定英文缩写。
$P(s' / s, a)$	Transition Probability	状态转移概率	从 s 执行 a 后到 s' 的概率。
$\pi(a / s)$	Policy	策略	在状态 s 选择动作 a 的概率。
$V_\pi(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的期望 return。
$Q_\pi(s, a)$	Action-value Function	动作价值函数	策略 π 下，在 s 先执行 a 的期望 return； Q 可理解为 action quality。
γ	Discount Factor	折扣因子	控制未来 reward 权重的系数。
RL	Reinforcement Learning	强化学习	本课程主题，学习长期决策策略。
POMDP	Partially Observable Markov Decision Process	部分可观测马尔可夫决策过程	观测不满足完整 Markov property 时的建模。

符号	English	中文	含义
TD	Temporal Difference	时序差分	后续用 Bellman target 做采样更新的方法。
DQN / PPO / A2C / SAC	Deep Q-Network / Proximal Policy Optimization / Advantage Actor-Critic / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 优势 Actor-Critic / 软 Actor-Critic	后续章节会学习的典型 deep RL 算法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	最基础的 Monte Carlo policy gradient 算法名。

本章目标

本章是强化学习的地基。你需要掌握：

- 强化学习和监督学习的区别。
- agent、environment、state、action、reward、policy 的含义。
- return 和 discount factor。
- MDP 五元组和 Markov property。
- $V(s)$ 、 $Q(s,a)$ 的定义和关系。
- Bellman expectation equation 与 Bellman optimality equation。

本章主线

本章先把 RL 问题压成一个标准数学对象：MDP。顺序是 agent 和 environment 交互，得到 trajectory；trajectory 产生 return；return 的期望定义出 $V_{\pi}(s)$ 和 $Q_{\pi}(s,a)$ ；value function 的自洽关系就是 Bellman equation。后续所有算法基本都在近似求解或优化这些 Bellman 关系。

强化学习在学什么

强化学习研究的是：一个 agent 通过和 environment 交互，学习一个 policy，使得长期累计 reward 最大。一次交互可以写成：

$$S_t \rightarrow A_t \rightarrow R_{t+1}, S_{t+1}$$

含义：

- S_t ：State at time t ，时刻 t 的状态。
- A_t ：Action at time t ，时刻 t 的动作。
- R_{t+1} ：Reward after action，执行 A_t 后得到的奖励。
- S_{t+1} ：Next state，环境转移到的下一个状态。

监督学习通常学习 $x \rightarrow y$ 的映射，而强化学习学习的是 sequential decision making。当前动作不仅影响当前 reward，也会影响未来会遇到的状态分布。

面试表达：

RL 的核心难点在于 delayed reward 和 sequential decision making。Agent 的动作会影响未来状态分布，因此数据不是固定的 i.i.d. 样本，目标也不是单步预测正确，而是长期累计收益最大。

Return

强化学习优化的是累计回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

也就是：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

这里 G_t 表示 Return，也就是“从时刻 t 开始往后看的累计回报”。 G 是强化学习教材中的常用记号，不建议把它理解成某个固定英文缩写。

γ 是 discount factor，通常在 $[0, 1]$ 。

γ 的作用：

- $\gamma = 0$ 时，agent 只关心眼前 reward。
- γ 接近 1 时，agent 更重视长期 reward。
- 在 infinite horizon 下，折扣可以让 return 更容易有界。
- 工程上， γ 也控制短视和长远之间的权衡。

面试追问：为什么需要 discount factor？

可以回答：

一方面是数学上让无限时域的累计 reward 更容易收敛，另一方面表达了对未来 reward 的偏好衰减或不确定性。 γ 越大越重视长期收益，但训练可能更难、更不稳定。

Policy

Policy 描述 agent 如何根据 state 选择 action。

确定性策略：

$$a = \pi(s)$$

随机策略：

$$\pi(a | s) = P(A_t = a | S_t = s)$$

强化学习最终通常是在寻找最优策略：

$$\pi^* = \operatorname{argmax}_{\pi} \mathbb{E}_{\pi}[G_t]$$

常见算法分类：

- value-based：先学习价值函数，再通过价值函数导出策略，例如 Q-learning、DQN。
- policy-based：直接参数化策略并优化，例如 REINFORCE、PPO。
- actor-critic：同时学习策略 actor 和价值 critic，例如 A2C、SAC。

MDP

MDP 是 Markov Decision Process，强化学习最常见的数学建模。

MDP 五元组：

$$(S, A, P, R, \gamma)$$

分别表示：

- S : State Space，状态空间。
- A : Action Space，动作空间。
- $P(s' | s, a)$: Transition Probability，状态转移概率。
- $R(s, a)$ 或 $R(s, a, s')$: Reward Function，奖励函数。
- γ : Discount Factor，折扣因子。

MDP 的核心是假设 Markov property：

$$\begin{aligned} P(S_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots) \\ = P(S_{t+1} | S_t, A_t) \end{aligned}$$

也就是说，当前 state 已经包含了预测未来所需的全部历史信息。

面试表达：

MDP 假设 state 是历史信息的 sufficient statistic。给定当前 state 和 action 后，未来状态转移与更早的历史无关。如果观测不满足这个条件，例如只能看到局部信息，则更准确的模型是 POMDP。

Value Function

状态价值函数（State-value Function，记作 V ， V 来自 value）：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

含义：从状态 s 出发，之后一直按照策略 π 行动，期望累计 return 是多少。

动作价值函数（Action-value Function，记作 Q ， Q 可理解为 action quality）：

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

含义：在状态 s 先执行动作 a ，之后按照策略 π 行动，期望累计 return 是多少。

两者关系：

$$V_{\pi}(s) = \sum_a \pi(a | s) Q_{\pi}(s, a)$$

最优情况下：

$$V^*(s) = \max_a Q^*(s, a)$$

最优动作：

$$a^* = \operatorname{argmax}_a Q^*(s, a)$$

为什么 Q 更适合直接决策？

因为 $Q(s, a)$ 已经评价了具体 action 的长期价值，可以直接用 $\operatorname{argmax}_a Q(s, a)$ 选动作。而 $V(s)$ 只评价状态好坏，如果不知道环境转移模型，单独的 $V(s)$ 不一定能直接告诉你该选哪个动作。

Bellman 方程

Return 可以递归拆开：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

于是价值函数也可以递归表示：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

展开得到 Bellman expectation equation：

$$\begin{aligned} V_{\pi}(s) &= \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_{\pi}(s')] \end{aligned}$$

对于动作价值函数：

$$\begin{aligned} Q_{\pi}(s, a) &= \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma \sum_{a'} \pi(a' | s') Q_{\pi}(s', a')] \end{aligned}$$

最优 Bellman 方程：

$$\begin{aligned} V^*(s) &= \max_a \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V^*(s')] \end{aligned}$$

$$Q^*(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]$$

这个式子非常重要。Q-learning 和 DQN 都是在用采样和函数逼近来逼近它。

Bellman 推导过程：不要直接背公式

Bellman 方程可以按四步讲。

第一步，从 return 的定义出发：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

把第一项单独拿出来，后面的部分正好是从 $t+1$ 开始的 return：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

第二步，对“从状态 s 出发”的长期回报取期望：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t=s]$$

把递归式代进去：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} | S_t=s]$$

第三步，把 G_{t+1} 换成下一状态的价值。因为到达 S_{t+1} 后，后续仍按策略 π 行动：

$$\mathbb{E}_{\pi}[G_{t+1} | S_{t+1}=s'] = V_{\pi}(s')$$

于是得到：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t=s]$$

第四步，把期望展开成“先按策略选动作，再按环境转移到下一个状态”：

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_{\pi}(s')]$$

这就是 Bellman expectation equation。

从 expectation 到 optimality 只改一个地方：不再对动作按当前策略求平均，而是假设在每个状态都选最优动作：

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

面试表达：

Bellman 方程本质是价值函数的递归自洽关系。当前状态的价值等于即时 reward 加上下一个状态价值的折扣期望。Expectation version 是评估固定策略，optimality version 是每一步都对动作取最大值。

从 Bellman 到算法的推理链

不同 RL 算法可以看成在回答同一个问题：怎么逼近 Bellman backup?

方法	推理方式	需要环境模型吗	用什么近似 Bellman target
DP	直接对 $P(s' s,a)$ 求和	需要	完整期望 backup
MC	采样完整 episode	不需要	实际 return G_t
TD	采样一步 transition	不需要	$r + \gamma V(s')$
Q-learning	采样一步 transition，并对下一动作取 max	不需要	$r + \gamma \max_a Q(s', a')$
DQN	用神经网络拟合 Q-learning target	不需要	$r + \gamma \max_a Q(s', a'; \theta)$

所以不要把这些算法看成完全无关的技巧。它们都围绕 Bellman target，只是期望怎么估、target 怎么构造、是否用函数逼近不同。

TD Target 与 TD Error

Q-learning 的更新式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

其中：

$$\begin{aligned} TD \text{ target} &= r + \gamma \max_{a'} Q(s', a') \\ TD \text{ error} &= TD \text{ target} - Q(s, a) \end{aligned}$$

直觉：

- 旧估计是 $Q(s,a)$ 。
- 新观测告诉我们，一步之后的目标应该是 $r + \gamma \max_{a'} Q(s', a')$ 。
- 两者差值就是 TD error。
- 用学习率 α 做一个增量更新。

本章例子：把问题建模成 MDP

例子 1：GridWorld 迷宫

一个机器人在网格里走，从起点走到终点。

MDP 元素	对应例子
State	机器人当前格子坐标，例如 (2,3)。
Action	上、下、左、右。
Transition	执行动作后到达相邻格子，也可能因为墙或噪声停在原地。
Reward	到终点 +10，撞墙 -1，每走一步 -0.1。
Policy	在每个格子选择哪个方向走。
Value	从某个格子出发，长期来看能拿到多少 reward。

这个例子的价值在于直观：离终点近、不容易撞墙的状态价值高；容易掉进惩罚区域的状态价值低。

例子 2：推荐系统

用户打开 App，系统要选择展示什么内容。

MDP 元素	对应例子
State	用户画像、最近点击、当前场景、时间、设备。
Action	推荐某个视频、商品或文章。
Reward	点击、停留、购买、负反馈等综合信号。
Transition	用户看完内容后兴趣状态变化。
Policy	给不同用户和场景选择不同推荐内容。

如果只看单次点击，这更像 bandit；如果考虑长期留存、疲劳、兴趣迁移，就更像完整 RL。

面试表达：

我会先问这个问题有没有长期影响。如果动作只影响当前 reward，可以先用 bandit；如果动作会改变后续状态，例如用户兴趣或库存状态，就需要 MDP/RL 视角。

本章面试高频问题

1. RL 和监督学习有什么区别？

监督学习有固定标签，通常假设样本 i.i.d.，目标是拟合输入到标签的映射。RL 没有逐步标准答案，只有 reward，且 reward 可能延迟出现。Agent 的动作会改变未来状态分布，因此需要处理 exploration-exploitation 和长期收益最大化。

2. 什么是 MDP？

MDP 是强化学习的标准建模，由状态空间、动作空间、转移概率、奖励函数和折扣因子组成。它的核心假设是 Markov property，即未来只依赖当前状态和动作，而不依赖完整历史。

3. V 和 Q 有什么区别?

$V_\pi(s)$ 衡量状态 s 在策略 π 下的长期价值。 $Q_\pi(s,a)$ 衡量在状态 s 先执行动作 a 后, 再跟随策略 π 的长期价值。 Q 多了动作维度, 因此更适合直接用来选动作。

4. Bellman expectation equation 和 Bellman optimality equation 区别?

Bellman expectation equation 用来评估一个给定策略 π 的价值。Bellman optimality equation 描述最优价值函数, 会在动作上取最大值, 对应寻找最优策略。

5. Bellman 方程的核心直觉是什么?

当前价值等于即时奖励加上折扣后的未来价值。它把长期决策问题拆成一步 reward 和下一状态 value 的递归组合。

本章练习

1. 用自己的话解释为什么强化学习的数据不是固定 i.i.d.。
2. 手写 $V_\pi(s)$ 、 $Q_\pi(s,a)$ 、 $V_\pi(s)$ 和 $Q_\pi(s,a)$ 的关系。
3. 从 $G_t = R_{t+1} + \gamma G_{t+1}$ 推导 Bellman expectation equation。
4. 解释为什么 $Q^*(s,a)$ 的 Bellman optimality equation 中有 $\max_a$ 。

第 2 章：DP（Dynamic Programming，动态规划）、Policy Evaluation 与 Value Iteration

本章符号说明

符号/缩写	English	中文	含义
DP	Dynamic Programming	动态规划	已知环境模型时，通过 Bellman backup 求解价值和策略。
$P(s' s, a)$	Transition Probability	状态转移概率	从 s 执行 a 后到 s' 的概率。
$R(s, a, s')$	Reward Function	奖励函数	从 s 执行 a 到 s' 时的奖励。
π / π_{new}	Policy / New Policy	策略 / 新策略	当前被评估或改进的策略。
$V_{\pi}(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_{\pi}(s, a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s, a) 的长期价值。
$V_k(s)$	Value Estimate at iteration k	第 k 次迭代的价值估计	value iteration / policy evaluation 中的中间估计。
γ	Discount Factor	折扣因子	控制未来 reward 权重。
GPI	Generalized Policy Iteration	广义策略迭代	policy evaluation 和 policy improvement 交替推进的通用框架。
MDP	Markov Decision Process	马尔可夫决策过程	本章 DP 要求解的对象。
RL	Reinforcement Learning	强化学习	DP 是 RL 的理论原型之一。
TD	Temporal Difference	时序差分	后续用采样近似 Bellman backup 的方法。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似 Q-learning 的算法。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	on-policy TD control 算法。

本章目标

本章学习在已知环境模型 P 和 R 的情况下，如何通过动态规划求解 MDP。

你需要掌握：

- Policy Evaluation。
- Policy Improvement。
- Policy Iteration。
- Value Iteration。
- Generalized Policy Iteration。
- DP 方法的局限性。

本章主线

本章回答一个理想化问题：如果 $P(s'/s, a)$ 和 $R(s, a, s')$ 都已知，Bellman equation 能不能直接求解？答案是肯定的。Policy Evaluation 负责“评估固定策略”，Policy Improvement 负责“按价值改进策略”，两者交替就是 GPI。第 3 章会把这里的精确 Bellman backup 换成采样 backup。

动态规划适用前提

动态规划类方法通常假设环境模型已知：

$$P(s' / s, a), R(s, a, s')$$

也就是说，agent 知道每个动作会以什么概率转移到什么状态，并得到什么 reward。

这在真实问题中经常不成立，但 DP 仍然重要，因为它给出了强化学习算法的理论原型。后面的 TD、Q-learning、DQN 都可以看成是在没有完整模型时，用采样近似 Bellman backup。

Policy Evaluation

Policy Evaluation 的目标：给定一个策略 π ，计算它的价值函数 $V_\pi(s)$ 。

Bellman expectation equation：

$$\begin{aligned} V_\pi(s) &= \sum_a \pi(a / s) \sum_{s'} P(s' / s, a) \\ &\quad [R(s, a, s') + \gamma V_\pi(s')] \end{aligned}$$

迭代更新：

$$\begin{aligned} V_{k+1}(s) &= \sum_a \pi(a / s) \sum_{s'} P(s' / s, a) \\ &\quad [R(s, a, s') + \gamma V_k(s')] \end{aligned}$$

直觉：

1. 初始化所有状态价值。
2. 使用 Bellman expectation backup 更新每个状态。
3. 重复直到价值函数收敛。

伪代码：

```

initialize V(s) arbitrarily
repeat:
    delta = 0
    for each state s:
        v_old = V(s)
        V(s) = sum_a pi(a|s) sum_s' P(s'|s,a) [r + gamma V(s')]
        delta = max(delta, |v_old - V(s)|)
until delta < threshold

```

Policy Improvement

有了 $V_{\pi}(s)$ 之后，可以通过贪心方式改进策略：

$$\pi_{new}(s) = \operatorname{argmax}_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_{\pi}(s')]$$

直觉：

如果在某个状态下，选择动作 a 后的期望回报比当前策略更好，就应该把策略改成选择 a 。

Policy Improvement Theorem：

如果对所有状态都有：

$$Q_{\pi}(s, \pi_{new}(s)) \geq V_{\pi}(s)$$

那么新策略不差于旧策略：

$$V_{\pi_{new}}(s) \geq V_{\pi}(s)$$

Policy Iteration

Policy Iteration 在两个步骤之间交替：

1. Policy Evaluation：评估当前策略。
2. Policy Improvement：根据价值函数改进策略。

流程：

```

initialize pi randomly
repeat:
    V_pi = policy_evaluation(pi)
    pi_new = greedy_policy(V_pi)
    if pi_new == pi:
        stop
    pi = pi_new

```

它最终会收敛到最优策略 π^* 。

面试表达：

Policy Iteration 是 evaluation 和 improvement 的交替过程。先精确或近似评估当前策略，再对价值函数做贪心改进。这个过程可以看成广义策略迭代的一种具体形式。

Value Iteration

Value Iteration 直接使用 Bellman optimality backup：

$$\begin{aligned} V_{k+1}(s) &= \max_a \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V_k(s')] \end{aligned}$$

和 Policy Iteration 的区别：

- Policy Iteration 每轮先完整或近似评估一个策略，再改进策略。
- Value Iteration 把 evaluation 和 improvement 合在一次 backup 中。
- Value Iteration 每次更新都直接对动作取 max。

最后从 V^* 导出策略：

$$\begin{aligned} \pi^*(s) &= \operatorname{argmax}_a \sum_{s'} P(s' | s, a) \\ &\quad [R(s, a, s') + \gamma V^*(s')] \end{aligned}$$

伪代码：

```
initialize V(s) arbitrarily
repeat:
    delta = 0
    for each state s:
        v_old = V(s)
        V(s) = max_a sum_s' P(s'|s,a) [r + gamma V(s')]
        delta = max(delta, |v_old - V(s)|)
until delta < threshold
derive pi from V
```

Generalized Policy Iteration

Generalized Policy Iteration，简称 GPI，指的是 policy evaluation 和 policy improvement 相互作用的一大类方法。

核心思想：

- evaluation 让 value function 更接近当前策略的真实价值。

- improvement 让 policy 根据当前 value function 变得更贪心。
- 两者交替推进，最终趋向最优策略和最优价值函数。

很多 RL 算法都可以放进这个框架：

- Policy Iteration：完整 evaluation + 贪心 improvement。
- Value Iteration：每一步都做截断 evaluation + improvement。
- SARSA：用采样做 on-policy evaluation 和 improvement。
- Q-learning：用采样逼近 optimal Bellman backup。
- Actor-Critic：critic 做 evaluation，actor 做 improvement。

DP 的局限性

动态规划方法有两个主要限制：

1. 需要已知完整环境模型 P 和 R 。
2. 状态空间和动作空间大时计算成本很高。

这也是为什么后续要学习 model-free RL：

- Monte Carlo 不需要环境模型，但要等 episode 结束。
- TD 不需要模型，也不需要等 episode 结束。
- Deep RL 用神经网络处理大规模状态空间。

本章例子：小型 GridWorld 上的 DP

假设一个 3x3 GridWorld：

- 右下角是终点，reward 为 +10。
- 撞墙留在原地，reward 为 -1。
- 普通移动 reward 为 -0.1。
- 每个动作都是确定性的。

Policy Evaluation 例子

如果当前策略是在每个格子随机选择上下左右，那么 policy evaluation 做的是：

固定这个随机策略，计算每个格子的长期价值。

直觉结果：

- 靠近终点的格子价值更高。
- 靠近墙角但远离终点的格子价值较低。
- 如果随机策略经常撞墙，撞墙附近的价值会被拉低。

这时算法只是在回答：“如果我一直按这个随机策略走，每个格子平均值多少钱？”

Policy Improvement 例子

有了 $V_{\pi}(s)$ 后，在某个格子比较四个动作：

向上：下一格价值 1.2
 向下：下一格价值 4.8
 向左：撞墙，价值 -1.0
 向右：下一格价值 3.5

policy improvement 会选择“向下”，因为它的一步 reward 加未来价值最大。

Value Iteration 例子

value iteration 不等完整 policy evaluation 收敛，而是每轮直接做：

每个格子都看四个动作，选 Bellman backup 最大的那个。

所以它更像“边估值，边贪心改策略”。在小 GridWorld 上，这会逐渐形成指向终点的箭头场。

面试表达：

DP 的例子一定要强调“已知模型”。GridWorld 里我知道每个动作会到哪个格子，所以能对下一状态做 Bellman backup。到了真实环境里不知道转移模型，就要转向 MC/TD。

本章面试高频问题

1. Policy Evaluation 是什么？

给定策略 π ，根据 Bellman expectation equation 计算该策略下每个状态的价值 $V_{\pi}(s)$ 。

2. Policy Iteration 和 Value Iteration 的区别？

Policy Iteration 交替进行策略评估和策略改进。Value Iteration 直接使用 Bellman optimality backup，把评估和改进合并在一次 max backup 中。

3. 为什么 DP 方法在真实问题中受限？

因为它通常要求环境模型已知，即知道完整的状态转移概率和奖励函数。而真实环境中模型往往未知，且状态动作空间可能极大。

4. Value Iteration 为什么能得到最优策略？

它反复应用 Bellman optimality operator。在折扣 MDP 中，该 operator 是 contraction mapping，因此价值函数会收敛到唯一的最优价值函数 V^* ，再由 V^* 导出最优策略。

5. GPI 是什么？

GPI 是 policy evaluation 和 policy improvement 交替推进的通用思想。很多强化学习算法都可以看成不同形式的 GPI。

本章练习

1. 手写 Policy Evaluation 的 Bellman update。
2. 对比 Policy Iteration 和 Value Iteration 的伪代码。
3. 解释为什么 Value Iteration 中可以不完整评估一个策略。
4. 思考：如果状态空间有一亿个状态，表格型 DP 为什么不可行？

第 3 章：MC（Monte Carlo，蒙特卡洛）、TD（Temporal Difference，时序差分）、SARSA 与 Q-learning

本章符号说明

符号/缩写	English	中文	含义
MC	Monte Carlo	蒙特卡洛	用完整 episode 的真实 return 估计价值。
TD	Temporal Difference	时序差分	用 bootstrap target 在线更新价值估计。
DP	Dynamic Programming	动态规划	已知环境模型时，通过 Bellman backup 求解价值和策略。
MDP	Markov Decision Process	马尔可夫决策过程	强化学习中描述状态、动作、转移、奖励和折扣的数学框架。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	更新公式使用的五元组名称。
$S_t / A_t / R_{t+1}$	State / Action / Reward	状态 / 动作 / 奖励	一步交互中的随机变量。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 到 episode 结束的累计收益。
$V(s)$	State-value Function	状态价值函数	状态 s 的长期价值估计。
$Q(s,a)$	Action-value Function	动作价值函数	状态 s 下动作 a 的长期价值估计。
δ_t	TD Error	时序差分误差	bootstrap target 和当前估计之间的差。
α	Learning Rate	学习率	控制每次更新幅度。
γ	Discount Factor	折扣因子	控制未来 reward 权重。
RL	Reinforcement Learning	强化学习	本章讨论 model-free RL 的估值和控制方法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	on-policy Monte Carlo policy gradient 算法。
DQN	Deep Q-Network	深度 Q 网络	off-policy value-based deep RL 算法。

符号/缩写	English	中文	含义
DDPG	Deep Deterministic Policy Gradient	深度确定性策略梯度	连续动作空间的 off-policy actor-critic 算法。
TD3	Twin Delayed DDPG	双延迟 DDPG	DDPG 的稳定化改进。
SAC	Soft Actor-Critic	软 Actor-Critic	最大熵 off-policy actor-critic 算法。
PPO	Proximal Policy Optimization	近端策略优化	on-policy actor-critic 算法。
A2C	Advantage Actor-Critic	优势 Actor-Critic	on-policy actor-critic 算法。

本章目标

本章进入 model-free RL。你需要掌握：

- Monte Carlo 方法。
- Temporal Difference Learning。
- MC 和 TD 的 bias/variance 区别。
- SARSA。
- Q-learning。
- on-policy 与 off-policy。

本章主线

第 2 章的 DP 依赖完整环境模型。本章去掉这个假设，只允许 agent 从真实交互中拿到样本。

主线分两步：

1. Prediction：先学会评估一个固定策略的价值。MC 用完整 return 做 target，TD 用一步 reward 加下一状态估计做 bootstrap target。
2. Control：再把价值估计用于改进策略。为了直接比较动作，需要从 $V(s)$ 转到 $Q(s,a)$ 。SARSA 和 Q-learning 不是突然出现的新主题，而是 TD 思想在 action-value control 上的两个版本：SARSA 是 on-policy TD control，Q-learning 是 off-policy TD control。

所以本章的逻辑是：

```
DP exact backup
  -> model-free sample backup
  -> MC/TD prediction
  -> TD control with Q(s,a)
  -> SARSA vs Q-learning
```

Model-free 的含义

Model-free 指的是不需要显式知道环境转移模型：

$$P(s' | s, a), R(s, a, s')$$

Agent 只通过和环境交互得到样本：

$$(s, a, r, s')$$

然后用这些样本估计价值函数或优化策略。

统一出发点：价值函数是一个期望

MC 和 TD 都不是凭空来的。它们解决的是同一个问题：价值函数定义里有一个期望，但 model-free 场景下我们没法直接算这个期望。

先看状态价值函数：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

其中 return 是：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

这句话的含义是：

状态 s 的价值，就是在策略 π 下从 s 出发，未来累计折扣 reward 的平均水平。

问题在于这个平均值不能直接看到。我们每次和环境交互，只能看到一条随机轨迹：

$$S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}, R_{t+2}, \dots$$

因此从价值定义到算法，核心只有一步：

用样本估计期望。

不同方法的区别在于“用什么样的样本 target”：

方法	可用信息	target 怎么来	代价
DP	知道 $P(s' s,a)$ 和 $R(s,a,s')$	对所有下一状态求精确期望	需要环境模型
MC	不知道模型，但能等 episode 结束	用完整真实 return G_t	等得久，方差大
TD	不知道模型，也不想等结束	用一步 reward 加下一状态估计 $R_{t+1} + \gamma V(S_{t+1})$	有 bootstrap bias

所以 MC 和 TD 可以理解成 DP 的两个 model-free 版本：

- MC：不用模型，也不用下一状态估计，直接等真实 return 出来。

- TD：不用模型，也不等真实 return，用一步采样加当前价值函数估计。

Monte Carlo 方法

Monte Carlo 方法用完整 episode 的实际 return 来估计价值。

对于状态价值：

```
V(s) <- average of returns observed after visiting s
```

增量形式：

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

其中 G_t 是从时间 t 到 episode 结束的真实累计 return。

特点：

- 不需要环境模型。
- 不需要 bootstrap。
- 必须等 episode 结束才能计算 return。
- 方差通常较大。
- 估计是无偏的，前提是采样足够充分。

First-visit MC：

每个 episode 中，一个状态第一次出现时才用它更新。

Every-visit MC：

每个 episode 中，一个状态每次出现都可以用来更新。

MC 的推理过程

MC 的出发点是价值函数定义：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

这个式子可以拆成一句统计学问题：

我想估计一个随机变量 G_t 在条件 $S_t = s$ 下的均值。

如果我们多次访问同一个状态 s ，就会得到很多个 return 样本：

$$G_t^{(1)}, G_t^{(2)}, \dots, G_t^{(n)}$$

用样本均值估计期望：

$$V_{\pi}(s) \approx 1/n \sum_{i=1}^n G_t^{(i)}$$

这就是 MC prediction 的核心。它不是先写更新公式，而是先把“价值函数”看成“return 的均值”。

如果每次都重新算平均值，写法是：

$$V_n(s) = 1/n \sum_{i=1}^n G_t^{(i)}$$

为了在线更新，可以把样本均值写成递推形式：

$$V_n(s) = V_{n-1}(s) + 1/n [G_t^{(n)} - V_{n-1}(s)]$$

如果不用严格的 $1/n$ ，而用固定学习率 α ，就得到常见写法：

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)]$$

这里的结构很重要：

```
new estimate = old estimate + step size * (target - old estimate)
```

对 MC 来说：

```
target = G_t
error = G_t - V(s)
```

因此 MC 的完整逻辑是：

1. 从状态 s 出发采样一条 episode。
2. 等 episode 结束后，计算真实 return G_t 。
3. 把 G_t 当作这次对 $V_\pi(s)$ 的观测样本。
4. 用样本平均或增量平均更新 $V(s)$ 。

为什么 MC target 是无偏的

$$\mathbb{E}_\pi[G_t | S_t=s] = V_\pi(s)$$

这说明在策略 π 不变、采样充分时， G_t 的期望就是目标价值。MC 用真实 return 做 target，target 本身不依赖当前估计 $V(s)$ ，因此没有 bootstrap bias。

但它的方差可能很大。原因是 G_t 包含整个未来：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

从 $S_t=s$ 开始，后面每一步动作、转移和 reward 都会引入随机性。轨迹越长，return 波动越大。

举个直觉例子：

- 同一个状态 s ，这次 episode 后面走到高奖励区域， G_t 很大。
- 下次 episode 后面因为探索动作走偏， G_t 很小。
- MC 会忠实记录这些完整结果，所以 target 真实，但抖动大。

First-visit 和 Every-visit 的区别

如果一个 episode 里同一个状态 s 出现多次：

```
s -> ... -> s -> ... -> terminal
```

First-visit MC 只用第一次访问后的 return 更新。Every-visit MC 每次访问都更新。

面试里一般不需要展开收敛证明，核心记住：

- 两者都在用 return 样本估计期望。
- first-visit 更贴近“每条 episode 给一个独立样本”的直觉。
- every-visit 使用更多样本，但同一 episode 内样本相关性更强。

从 MC 过渡到 TD：为什么不想等到 episode 结束

MC 的问题不是思路错，而是工程上有明显限制：

1. 很多任务 episode 很长，等完整 return 才更新太慢。
2. continuing task 没有自然终止， G_t 不容易等出来。
3. 长轨迹 return 方差大，学习曲线抖动。
4. 如果中间某一步已经明显好或坏，MC 也要等结束后才把信号传回前面状态。

TD 的动机就是解决这个问题：

能不能不等完整 G_t ，只走一步就更新？

答案来自 return 的递归分解：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

如果我们知道真实的 G_{t+1} ，那还是要等 episode 结束。但价值函数本身就是对 G_{t+1} 的估计：

$$V_{\pi}(S_{t+1}) = \mathbb{E}_{\pi}[G_{t+1} | S_{t+1}]$$

于是 TD 做了一个替换：

$$G_{t+1} \approx V(S_{t+1})$$

代回去：

$$G_t \approx R_{t+1} + \gamma V(S_{t+1})$$

这就是 TD target 的来源。

TD Learning

Temporal Difference Learning 结合了 Monte Carlo 和 Dynamic Programming 的思想。

TD(0) 更新：

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD target：

$$R_{t+1} + \gamma V(S_{t+1})$$

TD error：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

特点：

- 不需要环境模型。
- 不需要等 episode 结束。
- 使用 bootstrap，用当前估计 $V(S_{t+1})$ 更新当前估计。
- 通常方差低于 MC，但可能有 bias。

TD 的推理过程

TD 可以从两个等价角度推出来。

角度一：从 Bellman expectation 推出

Bellman expectation equation 是：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t = s]$$

如果有环境模型，DP 可以把这个期望展开成对所有动作和下一状态求和：

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V_{\pi}(s')]$$

但 model-free 场景下，我们不知道 $P(s'|s,a)$ ，也不知道完整的转移分布。我们只有一次真实采样：

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

于是用这一次样本近似 Bellman 期望里的随机变量：

$$R_{t+1} + \gamma V(S_{t+1})$$

这就是 stochastic approximation，即用随机样本近似期望更新。

角度二：从 return 递归推出

return 有递归形式：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

MC 使用真实 G_t ，所以必须等未来全部 reward 出来。TD 不等 G_{t+l} ，而是用当前估计替代：

$$G_{t+l} \approx V(S_{t+l})$$

所以 TD target 变成：

$$R_{t+l} + \gamma V(S_{t+l})$$

这一步叫 bootstrap，因为 target 里用了模型自己当前的估计。

TD error 的含义

有了 target 后，再用“target 减当前估计”构造误差：

$$\delta_t = R_{t+l} + \gamma V(S_{t+l}) - V(S_t)$$

然后按误差方向更新：

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t$$

如果 $\delta_t > 0$ ，说明当前 $V(S_t)$ 低估了，因为一步 reward 加下一状态价值比当前估计更高。

如果 $\delta_t < 0$ ，说明当前 $V(S_t)$ 高估了。

TD 为什么方差低但有 bias

TD target 只包含一步真实 reward：

$$R_{t+l}$$

后面的长期未来不再用真实随机轨迹，而用估计值：

$$V(S_{t+l})$$

因此 TD target 少受后续随机动作和随机转移影响，方差通常低于 MC。

但 $V(S_{t+l})$ 可能本身不准。target 依赖当前估计，所以会引入 bootstrap bias。

一句话：

MC 的 target 真实但晚、噪声大；TD 的 target 来得早、噪声小，但里面混入了当前价值函数的估计误差。

MC vs TD

维度	Monte Carlo	TD
是否需要模型	不需要	不需要

维度	Monte Carlo	TD
是否需要完整 episode	需要	不需要
是否 bootstrap	否	是
目标	真实 return G_t	$r + \gamma V(s')$
方差	较高	较低
偏差	较低	有 bootstrap bias
适用场景	episodic task	episodic 或 continuing task
更新时机	episode 结束后	每一步之后
信息传播	等完整 return 再回传	reward 可以一步步向前传播

面试表达：

MC 用完整轨迹的真实 return 做 target，不 bootstrap，所以 bias 小但 variance 大。TD 用一步 reward 加下一状态的估计值做 target，能在线更新，variance 小，但因为 target 依赖当前估计，所以会引入 bias。

一个最小数值例子

假设当前估计：

$$V(s)=5, V(s')=8, \gamma=0.9, \alpha=0.1$$

这一步从 s 到 s' ，拿到 reward：

$$R_{t+1}=2$$

TD target 是：

$$2+0.9 \times 8=9.2$$

TD error 是：

$$\delta=9.2-5=4.2$$

更新后：

$$V(s) \leftarrow 5+0.1 \times 4.2=5.42$$

这里没有等 episode 结束。TD 只用了一步真实 reward 和下一状态当前估计。

如果用 MC，则必须等整条 episode 结束。假设最后算出来真实 return 是：

$$G_t = 12$$

MC 更新是：

$$V(s) \leftarrow 5 + 0.1 \times (12 - 5) = 5.7$$

两者差异就在 target：

- MC target 是完整真实 return：12。
- TD target 是一步 bootstrap：9.2。

这个例子说明：TD 不一定比 MC 更准，它只是更早、更低方差地更新。

本章例子：什么时候用 MC，什么时候用 TD

例子 1：Blackjack 适合解释 MC

Blackjack 是一个 episodic task：

- 一局牌开始。
- 玩家要牌或停牌。
- 最后赢、输或平局。
- episode 很短，结束明确。

这时 MC 很自然。比如玩家在某个状态 s 做决策后，等这一局结束就能知道真实结果：

```
赢了: G_t = +1
输了: G_t = -1
平局: G_t = 0
```

多玩很多局后，访问同一个状态得到很多 return 样本，用平均值估计这个状态价值。

面试讲法：

Blackjack 适合 MC，因为 episode 短、终止清晰、完整 return 容易拿到。MC 不需要知道牌堆转移概率，只要反复采样完整对局。

例子 2：在线广告更适合解释 TD

假设系统每次给用户推荐一个广告，用户状态会持续变化，没有明确“游戏结束”。

如果用 MC，可能要等用户一整天甚至一周的长期收益才更新，这太慢。

TD 可以每一步更新：

当前状态：用户刚打开 App
 动作：展示广告 A
 即时 reward：点击 +1，没有点击 0
 下一状态：用户继续浏览或离开
 TD target = 即时 reward + 下一状态价值估计

这说明 TD 的优势：

- 不需要等完整长期结果。
- 每次交互后都能更新。
- 适合 continuing task 或长 episode 任务。

从 Prediction 到 Control：为什么接 SARSA 和 Q-learning

到这里为止，MC 和 TD 主要解决的是 prediction：

给定一个策略 π ，估计它的价值。

但强化学习最终要做 control：

找到一个更好的策略。

两者的区别是：

任务	学什么	策略是否固定	典型问题
V prediction	$V_{\pi}(s)$	固定	当前策略下，状态 s 值多少钱
Q prediction	$Q_{\pi}(s,a)$	固定	当前策略下，在 s 做 a 值多少钱
control	$Q(s,a)$ 并改进 π	不固定	在 s 应该选哪个动作

为什么 control 常从 $Q(s,a)$ 入手？

如果已知环境模型，可以用 $V(s)$ 加上 $P(s'|s,a)$ 和 $R(s,a,s')$ 做一步 lookahead：

$$\sum_s P(s'|s,a) [R(s,a,s') + \gamma V(s')]$$

但 model-free 场景下，模型未知，agent 不能直接对所有下一状态求和。此时更自然的是直接学习每个动作的长期价值：

$$Q(s,a)$$

有了 $Q(s,a)$ ，策略改进就很直接：

choose action with larger $Q(s, a)$

所以 SARSA 和 Q-learning 的位置应该理解为：

```
TD prediction learns  $V_{\pi}(s)$ 
TD control learns  $Q(s,a)$  and improves policy
SARSA and Q-learning are two TD control algorithms
```

它们都沿用 TD 的更新骨架：

```
new estimate = old estimate + alpha * (TD target - old estimate)
```

区别只在 TD target 怎么定义：

- SARSA：target 跟随实际行为策略采样到的下一个动作。
- Q-learning：target 假设下一步执行 greedy action。

TD Control 1: SARSA

SARSA 是 on-policy TD control 方法。名字来自一次更新用到的五元组：

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$$

更新公式：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

关键点：

- A_{t+1} 是当前行为策略实际选择的动作。
- 如果行为策略是 epsilon-greedy，那么 SARSA 的 target 也包含探索动作的影响。
- 因此 SARSA 是 on-policy。

直觉：

SARSA 学的是“我按照当前这套带探索的策略行动，长期回报会是多少”。

SARSA 的 target 为什么是 $Q(s',a')$

对 action-value 来说，Bellman expectation 可以理解成：

$$Q_{\pi}(s,a) = \mathbb{E}[R_{t+1} + \gamma Q_{\pi}(S_{t+1}, A_{t+1})]$$

SARSA 在采样时真的执行了下一步动作 A_{t+1} 。如果当前行为策略是 epsilon-greedy，那么 A_{t+1} 可能是 greedy action，也可能是探索动作。

所以 SARSA 的 target 是：

$$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$$

它把“探索可能造成的后果”也学进去。这就是为什么在 cliff walking 里 SARSA 更保守：它知道自己未来还可能因为 epsilon 探索而掉下去。

TD Control 2: Q-learning

Q-learning 是 off-policy TD control 方法。

更新公式：

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

关键点：

- 行为时可以用 epsilon-greedy 探索。
- 更新 target 时直接用 $\max_a Q(s', a)$ 。
- 学习目标是 greedy policy 的价值，而不完全等于行为策略。
- 因此 Q-learning 是 off-policy。

直觉：

Q-learning 学的是“如果未来都选当前估计下最优的动作，长期回报会是多少”。

Q-learning 的 target 为什么用 max

Q-learning 对应的是 Bellman optimality 思路：

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_a Q^*(S_{t+1}, a')]$$

model-free 场景下，把期望用一次 transition 近似，就得到 target：

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a')$$

这里要分清两件事：

- 行为时：可以用 epsilon-greedy 收集数据，保留探索。
- 学习时：target 假设未来执行 greedy action。

所以 Q-learning 是 off-policy。它不是在评估“实际带探索的行为策略”，而是在用探索数据学习一个 greedy target policy 的价值。

面试里可以这样推：

SARSA 从 Bellman expectation 来，下一动作就是行为策略实际采样的 a' ；Q-learning 从 Bellman optimality 来，下一动作取当前 Q 最大的动作。因此 SARSA on-policy，Q-learning off-policy。

SARSA vs Q-learning

维度	SARSA	Q-learning
策略类型	on-policy	off-policy
target	$r + \gamma Q(s', a')$	$r + \gamma \max_a Q(s', a)$
是否考虑探索动作影响	是	否
风格	更保守	更贪心
学习目标	当前行为策略	最优 greedy 策略

本章例子：Cliff Walking 解释 SARSA 和 Q-learning

Cliff Walking 中，起点到终点的最短路径贴着悬崖走，但 epsilon-greedy 探索可能让 agent 随机走错一步掉下去。

SARSA 学当前带探索行为策略的价值，所以它会把“未来仍可能探索并掉下悬崖”的风险计入 target：

```
r + gamma * Q(s', actual a')
```

因此 SARSA 往往学到更保守、更远离悬崖的安全路径。

Q-learning 学 greedy target policy 的价值。虽然行为时仍用 epsilon-greedy 探索，但更新 target 假设下一步会选择当前最优动作：

```
r + gamma * max_a Q(s', a)
```

因此 Q-learning 更容易学到贴近悬崖的最短路径。

这个例子对应的逻辑是：

- SARSA 会考虑 epsilon-greedy 探索可能掉下悬崖，因此倾向选择更安全路径。
- Q-learning 更新时假设未来总能选最优动作，因此更倾向学习贴近悬崖的最短路径。

面试讲法：

SARSA 和 Q-learning 都是 TD control。SARSA 的 target 来自行为策略实际采样到的下一动作，所以是 on-policy；Q-learning 的 target 来自 greedy max，所以是 off-policy。Cliff Walking 中 SARSA 更保守，Q-learning 更贪心。

On-policy vs Off-policy

On-policy：

学习和评估的是当前实际采样的行为策略。

$$\pi_{behavior} = \pi_{target}$$

例如：

- SARSA
- REINFORCE
- A2C
- PPO

Off-policy：

用一种行为策略收集数据，但学习另一种目标策略。

$$\pi_{behavior} \neq \pi_{target}$$

例如：

- Q-learning
- DQN
- DDPG
- TD3
- SAC

off-policy 的优势：

- 可以复用历史数据。
- sample efficiency 通常更高。
- 可以从 replay buffer 学习。

off-policy 的难点：

- 分布不匹配。
- 重要性采样或 value correction 可能带来高方差。
- 结合函数逼近和 bootstrap 时可能不稳定。

本章面试高频问题

1. MC 和 TD 的区别？

MC 用完整 episode 的真实 return 更新，不 bootstrap，bias 小但 variance 大。TD 用 $r + \gamma V(s')$ 这种一步 bootstrap target 更新，可以在线学习，variance 小但有 bias。

2. SARSA 和 Q-learning 的区别？

SARSA 是 on-policy，target 使用实际采样到的下一个动作 a' 。Q-learning 是 off-policy，target 使用 $\max_a Q(s', a)$ ，学习 greedy target policy。

3. SARSA/Q-learning 和 TD 有什么关系？

SARSA 和 Q-learning 都是 TD control。TD prediction 更新 $V(s)$ ，用于评估固定策略；SARSA/Q-learning 更新 $Q(s,a)$ ，用于边估值边改进策略。两者都使用一步 bootstrap target，只是 SARSA 用实际采样的 a' ，Q-learning 用 greedy max。

4. 为什么 Q-learning 是 off-policy?

因为采样数据可以来自 epsilon-greedy 行为策略，但更新目标是假设下一步选择 greedy action 的价值，即 target policy 和 behavior policy 不同。

5. TD error 是什么?

TD error 是当前价值估计和 bootstrap target 之间的差：

$$\delta = r + \gamma V(s') - V(s)$$

它衡量当前估计需要被修正的方向和大小。

6. 为什么 TD 可以不等 episode 结束?

因为 TD target 只需要一步 reward 和下一状态的当前价值估计，不需要完整轨迹 return。

本章练习

1. 手写 SARSA 和 Q-learning 的更新公式。
2. 解释为什么 SARSA 在 cliff walking 中更保守。
3. 写一个 tabular Q-learning 的伪代码。
4. 用 bias/variance 角度比较 MC 和 TD。

第 4 章：探索与利用、Bandit、UCB (Upper Confidence Bound, 置信上界) 与 Thompson Sampling

本章符号说明

符号/缩写	English	中文	含义
UCB	Upper Confidence Bound	置信上界	用价值估计加不确定性 bonus 做探索。
K	Number of Arms	拉杆数量 / 动作数量	K-armed bandit 中候选动作的数量。
A_t	Action at time t	时刻 t 的动作	bandit 或 RL 中当前选择的动作。
R_t	Reward at time t	时刻 t 的奖励	本轮选择动作后获得的奖励。
$Q_t(a)$	Action-value Estimate	动作价值估计	到时刻 t 为止动作 a 的平均 reward 估计。
$N_t(a)$	Action Count	动作选择次数	到时刻 t 为止动作 a 被选过多少次。
μ_a / μ^*	Mean Reward / Best Mean Reward	平均奖励 / 最优平均奖励	动作 a 或最优动作的期望奖励。
$Regret(T)$	Cumulative Regret	累计后悔值	到时间 T 为止因为没有总选最优动作损失的收益。
τ	Temperature	温度参数	softmax exploration 中控制分布平滑程度。
RL	Reinforcement Learning	强化学习	探索与利用是 RL 的核心问题之一。
DQN	Deep Q-Network	深度 Q 网络	常用 epsilon-greedy 做探索的 value-based 算法。

本章目标

本章讨论强化学习中的 exploration-exploitation tradeoff。你需要掌握：

- 为什么需要探索。
- epsilon-greedy。
- softmax exploration。
- Multi-Armed Bandit。
- UCB。

- Thompson Sampling。
- regret。

本章主线

第 3 章默认 agent 能持续拿到交互样本，但样本本身由策略决定。策略如果只选当前最优动作，可能永远发现不了更好动作；如果一直随机试，又会牺牲收益。本章把探索问题抽出来，先在 bandit 里研究最小版本，再把 epsilon-greedy、UCB、Thompson Sampling 的思想带回 RL。

探索与利用

强化学习中，agent 面临两个目标：

- exploitation：选择当前看起来最好的动作，最大化已知收益。
- exploration：尝试不确定的动作，发现潜在更好的选择。

如果只 exploitation，可能永远错过更优动作。

如果只 exploration，则无法稳定获得高收益。

面试表达：

Exploration-exploitation tradeoff 是 RL 的核心问题之一。Agent 需要在利用当前价值估计和探索未知动作之间平衡。探索不足会导致局部最优，探索过多会损害短期收益和训练稳定性。

Epsilon-greedy

epsilon-greedy 是最常见的探索策略。

```
with probability epsilon:
    choose random action
otherwise:
    choose argmax_a Q(s, a)
```

优点：

- 简单。
- 容易实现。
- DQN 等算法常用。

缺点：

- 随机探索没有方向。
- 对所有非 greedy 动作一视同仁。
- 在大动作空间中效率较低。

常见工程做法：

```
epsilon starts high and decays over time
```

例如从 1.0 逐渐衰减到 0.05。

Softmax Exploration

softmax exploration 按照 action-value 的 softmax 概率采样动作：

$$P(a | s) = \exp(Q(s,a) / \tau) / \sum_b \exp(Q(s,b) / \tau)$$

τ 是 temperature：

- τ 大：分布更平，探索更多。
- τ 小：分布更尖，接近 greedy。

相比 epsilon-greedy，softmax 会更偏向价值较高的动作，而不是均匀随机探索。

缺点：

- 对 Q 值尺度敏感。
- 如果 Q 值估计不准，采样分布也会被误导。

Multi-Armed Bandit

Bandit 是最简单的 sequential decision 问题：

- 只有动作，没有状态转移。
- 每个 action 对应一个未知 reward distribution。
- 目标是在不断试验中最大化累计 reward。

K-armed bandit：

```
choose action A_t in {1, ..., K}
receive reward R_t
```

它可以看成没有状态或只有单一状态的 RL 问题。

Bandit 在推荐系统、广告投放、A/B testing 中非常常见。

Regret

Regret 衡量由于没有总是选择最优动作而损失的收益。

如果最优动作的期望奖励是 μ^* ，第 t 步选择动作的期望奖励是 μ_{a_t} ，则累计 regret：

$$\text{Regret}(T) = \sum_{t=1}^T (\mu^* - \mu_{a_t})$$

目标不是只最大化 reward，也可以表述为最小化 regret。

UCB

Upper Confidence Bound, 简称 UCB。

核心思想：

乐观面对不确定性。对不确定的动作给一个探索 bonus。

常见 UCB 形式：

$$A_t = \operatorname{argmax}_a [Q_t(a) + c \sqrt{(\ln t / N_t(a))}]$$

其中：

- $Q_t(a)$ ：动作 a 当前平均 reward 估计。
- $N_t(a)$ ：动作 a 被选择次数。
- t ：总时间步。
- c ：控制探索强度。

解释：

- 如果动作被选得少， $N_t(a)$ 小，bonus 大。
- 如果动作被选得多，不确定性下降，bonus 变小。
- UCB 会系统性探索不确定动作，而不是随机探索。

Thompson Sampling

Thompson Sampling 是一种 Bayesian exploration 方法。

流程：

1. 为每个动作维护 reward 分布参数的 posterior。
2. 每轮从每个动作的 posterior 中采样一个可能的参数。
3. 选择采样后看起来最优的动作。
4. 根据实际 reward 更新 posterior。

直觉：

一个动作的不确定性越大，它在采样中偶尔成为最优的概率越大，因此会自然获得探索机会。

优点：

- 探索非常自然。
- 在 bandit 问题中表现强。
- 常用于推荐、广告、实验分流。

缺点：

- 需要构造合适的概率模型。
- 在复杂深度 RL 中实现和建模成本更高。

NoisyNet 与参数空间探索

深度 RL 中还可以在参数空间探索。例如 NoisyNet 在网络参数中加入可学习噪声。

直觉：

- epsilon-greedy 是 action-level 随机。
- 参数空间噪声会让整个策略在一段时间内保持一致的探索模式。

这在需要 temporally coherent exploration 的任务中更合理。

本章例子：广告推荐里的探索

例子 1：Epsilon-greedy

假设有 3 个广告候选：

广告	当前点击率估计
A	4.0%
B	3.5%
C	1.0%

如果完全 exploitation，系统总是展示 A。但 C 的估计可能只是因为曝光少，不代表真实差。

epsilon-greedy 的做法：

- 95% 概率展示当前最优广告 A。
- 5% 概率随机探索 A/B/C。

这能避免过早把 B 或 C 判死。

例子 2：UCB

UCB 会给曝光少、不确定性高的广告更高 bonus。

直觉：

`score = 当前平均收益 + 不确定性奖励`

如果广告 C 曝光很少，即使平均点击率暂时低，也可能因为不确定性高被选中继续探索。

例子 3：Thompson Sampling

Thompson Sampling 会为每个广告维护一个后验分布，然后从分布里采样。

如果广告 A 数据多，后验分布窄；广告 C 数据少，后验分布宽。C 偶尔采样出很高的潜在点击率，就会获得探索机会。

面试表达：

Epsilon-greedy 是固定比例随机探索；UCB 是乐观面对不确定性；Thompson Sampling 是按后验不确定性自然采样。推荐和广告场景里，探索机制直接影响冷启动和长期收益。

本章面试高频问题

1. 为什么强化学习需要探索？

因为 agent 初始并不知道哪些动作长期收益最高。如果只选择当前估计最优的动作，可能因为早期估计错误而陷入局部最优。

2. epsilon-greedy 的缺点是什么？

探索是均匀随机的，没有利用不确定性信息，也不区分非 greedy 动作的潜在价值。在大动作空间中效率较低。

3. UCB 的核心思想是什么？

UCB 使用当前价值估计加上不确定性 bonus 选动作。被尝试较少的动作有更大 bonus，从而获得探索机会。

4. Thompson Sampling 和 UCB 的区别？

UCB 基于置信上界，显式构造 optimism bonus。Thompson Sampling 是 Bayesian 方法，从 posterior 采样参数，通过概率匹配自然平衡探索和利用。

5. Bandit 和完整 RL 的区别？

Bandit 没有复杂状态转移，动作只影响当前 reward。完整 RL 中动作会影响未来状态分布和长期 return。

本章练习

1. 写出 epsilon-greedy 的动作选择逻辑。
2. 解释 UCB 中 $\sqrt{(\ln t) / N(a)}$ 的直觉。
3. 用推荐系统举一个 bandit 的例子。
4. 比较 epsilon-greedy、UCB 和 Thompson Sampling。

第 5 章：函数逼近、稳定性与 Deadly Triad

本章符号说明

符号/缩写	English	中文	含义
$V(s)$	State-value Function	状态价值函数	状态 s 的长期价值。
$Q(s,a)$	Action-value Function	动作价值函数	状态-动作对 (s,a) 的长期价值。
$\pi(a s)$	Policy	策略	状态 s 下选择动作 a 的概率。
θ / w	Parameters / Weights	参数 / 权重	策略网络或价值网络的可学习参数。
y	Target	训练目标值	Bellman target，用来监督当前估计。
$L(w)$	Loss Function	损失函数	训练价值网络时最小化的目标。
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似 $Q(s,a)$ 的 value-based 算法。
TD	Temporal Difference	时序差分	用一步 reward 和下一状态估计构造 target 的方法。
RL	Reinforcement Learning	强化学习	本章讨论 RL 从表格方法进入函数逼近后的稳定性问题。
LLM	Large Language Model	大语言模型	文中作为高维状态或复杂序列系统的例子。

本章目标

本章从表格型 RL 过渡到深度强化学习。你需要掌握：

- 为什么需要函数逼近。
- 线性函数逼近和神经网络逼近。
- value function approximation。
- bootstrapping 的影响。
- off-policy learning 的分布问题。
- Deadly Triad。

本章主线

前面几章默认可以为每个状态或状态-动作对存一个表格值。真实任务里状态空间通常巨大或连续，所以必须用参数化函数近似 $V(s)$ 、 $Q(s,a)$ 或 $\pi(a|s)$ 。本章先建立函数逼近的训练形式，再解释为什么 bootstrapping、off-policy learning 和 function approximation 同时出现时会带来稳定性问题。

为什么需要函数逼近

表格型方法为每个状态或状态动作对存一个值：

$$V(s)$$

$$Q(s, a)$$

但真实任务中状态空间可能极大，甚至连续：

- 图像输入。
- 机器人连续状态。
- 推荐系统用户特征。
- LLM 对话状态。

这时无法为每个状态单独建表，需要用参数化函数近似价值函数或策略。

例如：

$$V(s) \approx V(s; w)$$

$$Q(s, a) \approx Q(s, a; w)$$

$$\pi(a | s) \approx \pi(a | s; \theta)$$

Value Function Approximation

用神经网络近似 Q 函数：

$$Q(s, a; w)$$

训练目标通常来自 Bellman target。

例如 DQN 中：

$$y = r + \gamma \max_{a'} Q(s', a'; w^-)$$

损失：

$$L(w) = \mathbb{E}[(y - Q(s, a; w))^2]$$

其中 w^- 是 target network 参数。

函数逼近的好处

- 可以处理高维状态。
- 可以泛化到没见过的状态。
- 可以结合深度学习特征提取能力。
- 能用于图像、文本、连续控制等任务。

函数逼近的风险

表格型方法中，不同状态动作对的值相互独立。

函数逼近中，共享参数会导致一个样本更新影响许多状态动作对的估计。

这带来几个问题：

- 训练目标非平稳。
- 样本之间高度相关。
- 误差可能通过 bootstrap 扩散。
- 价值函数可能过估计或发散。

Bootstrapping

Bootstrapping 指 target 中包含当前模型自己的估计。

例如 TD target：

$$r + \gamma V(s')$$

Q-learning target：

$$r + \gamma \max_{a'} Q(s', a')$$

它的优点：

- 可以在线更新。
- 不需要完整 episode。
- sample efficiency 较高。

它的风险：

- target 依赖当前估计。
- 估计错误会被继续传播。
- 函数逼近下更容易放大不稳定性。

Off-policy Distribution Shift

off-policy 学习中，数据来自 behavior policy，但学习目标是 target policy。

data distribution: $d_{\mu}(s, a)$

target policy: π

如果 behavior policy 产生的数据分布和 target policy 需要的数据分布差异很大，模型可能会在没数据覆盖的状态动作上产生错误估计。

这种问题在 offline RL 中尤其严重。

Deadly Triad

Deadly Triad 指三个因素同时出现时，RL 训练可能不稳定甚至发散：

1. Function approximation。
2. Bootstrapping。
3. Off-policy learning。

为什么危险？

- 函数逼近会让误差泛化到其他状态。
- bootstrapping 会用当前估计构造 target，错误会自我强化。
- off-policy 会带来数据分布和学习目标不一致。

DQN 同时包含这三个因素，因此需要额外稳定技巧：

- replay buffer 打破样本相关性。
- target network 稳定 bootstrap target。
- reward clipping。
- gradient clipping。
- Double DQN 降低 overestimation bias。

Experience Replay 的稳定作用

Replay buffer 存储历史 transition：

```
(s, a, r, s', done)
```

训练时从 buffer 中随机采样 mini-batch。

好处：

- 打破连续样本相关性。
- 提高样本利用率。
- 让数据分布更平滑。

限制：

- 数据可能 stale。
- 对 on-policy 算法不天然适用。
- 分布偏移仍然存在。

Target Network 的稳定作用

如果 target 和 online prediction 使用同一个网络：

$$y = r + \gamma \max_{a'} Q(s', a'; w)$$

那么模型参数一更新，target 也立刻变化，训练目标非常不稳定。

Target network 使用滞后的参数 w^- ：

$$y = r + \gamma \max_{a'} Q(s', a'; w^-)$$

每隔一段时间同步：

$$w^- \leftarrow w$$

或者软更新：

$$w^- \leftarrow \tau w + (1 - \tau) w^-$$

本章例子：为什么表格法不够

例子 1：Atari 像素输入

Atari 游戏状态是一张图片。如果用表格法，每一种像素组合都对应一个状态，状态数量几乎无限。

函数逼近的做法是：

输入：游戏画面
神经网络：CNN / encoder
输出：每个动作的 Q 值

这样模型可以泛化。例如两个画面只差几个像素，但局面本质相似，网络可以给出相近价值。

例子 2：推荐系统高维状态

推荐系统状态可能包含：

- 用户画像。
- 最近点击序列。
- 商品特征。
- 时间、地点、设备。

这些状态无法枚举。函数逼近可以把高维特征映射到价值或策略：

$$Q(s, a; \theta)$$

例子 3：Deadly Triad 为什么危险

假设用 DQN 训练一个推荐策略：

- function approximation：用神经网络估计 Q。
- bootstrapping：target 里有 $\max_a Q(s', a)$ 。
- off-policy：数据来自旧策略或 replay buffer。

如果新策略常推荐训练数据里很少出现的内容，Q 网络可能对这些动作过度乐观；bootstrap 又会把这种高估继续传播，训练就可能发散。

面试表达：

Deadly Triad 的危险不是三个词各自危险，而是它们叠加后，错误估计会通过 bootstrap 在 off-policy 分布外被不断放大。

本章面试高频问题

1. 为什么表格型 RL 不够用？

因为真实任务的状态空间通常很大或连续，无法为每个状态动作对单独存一个值。函数逼近可以泛化到未见状态，并处理高维输入。

2. 什么是 Deadly Triad？

Function approximation、bootstrapping 和 off-policy learning 三者同时出现时，强化学习训练容易不稳定甚至发散。

3. 为什么神经网络加 Q-learning 会不稳定？

Q-learning 使用 bootstrap target，target 又依赖当前估计。神经网络共享参数，一个样本更新会影响大量状态动作估计。再加上 off-policy 数据分布偏移，误差可能被放大。

4. replay buffer 解决什么问题？

它通过随机采样历史 transition，打破连续样本相关性，提高样本利用率，并让训练数据分布更平滑。

5. target network 解决什么问题？

它让 Bellman target 在一段时间内保持相对稳定，避免 online network 更新时 target 也同步剧烈变化。

本章练习

1. 用自己的话解释 bootstrapping。
2. 画出 DQN 中 online network 和 target network 的关系。
3. 解释为什么 off-policy + function approximation 容易出问题。
4. 总结 DQN 针对 Deadly Triad 做了哪些稳定化设计。

第 6 章：DQN（Deep Q-Network，深度 Q 网络）与经典改进

本章符号说明

符号/缩写	English	中文	含义
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似动作价值函数 $Q(s,a)$ 。
$Q(s,a)$	Action-value Function	动作价值函数	状态 s 下动作 a 的长期价值； Q 表示 action quality。
y	TD Target / Bellman Target	TD 目标 / Bellman 目标	用于监督当前 Q 网络的目标值。
$L(\theta)$	Loss Function	损失函数	网络参数 θ 的训练损失。
θ	Online Network Parameters	在线网络参数	当前正在训练的 Q 网络参数。
θ^-	Target Network Parameters	目标网络参数	滞后更新的 target network 参数。
p_i	Priority	优先级	Prioritized Replay 中第 i 条样本的采样优先级。
$P(i)$	Sampling Probability	采样概率	replay buffer 中样本 i 被采样的概率。
N	Buffer Size / Sample Count	buffer 大小 / 样本数	importance sampling weight 中的数据规模。
w_i	Importance Sampling Weight	重要性采样权重	修正非均匀采样 bias 的权重。
α / β	Priority Exponent / IS Correction Exponent	优先级指数 / 重要性采样修正指数	Prioritized Replay 中控制采样偏斜和 bias 修正的参数。
TD	Temporal Difference	时序差分	DQN 使用 TD/Bellman target 训练 Q 网络。
RL	Reinforcement Learning	强化学习	DQN 是 deep RL 的经典 value-based 算法。
SGD	Stochastic Gradient Descent	随机梯度下降	用 mini-batch 优化神经网络的训练方法。

符号/缩写	English	中文	含义
IS	Importance Sampling	重要性采样	Prioritized Replay 中用于修正非均匀采样 bias。
DDPG / TD3 / SAC / PPO	Deep Deterministic Policy Gradient / Twin Delayed DDPG / Soft Actor-Critic / Proximal Policy Optimization	深度确定性策略梯度 / 双延迟 DDPG / 软 Actor-Critic / 近端策略优化	连续控制或策略优化章节会出现的算法。

本章目标

本章学习 value-based deep RL 的代表算法 DQN。你需要掌握：

- DQN 的基本结构。
- experience replay。
- target network。
- Double DQN。
- Dueling DQN。
- Prioritized Experience Replay。
- DQN 的适用场景和局限。

本章主线

本章把第 3 章的 Q-learning 和第 5 章的神经网络函数逼近接起来。直接用网络拟合 Q-learning target 会因为样本相关、target 移动、过估计等问题不稳定。DQN 的 replay buffer、target network、Double DQN、Dueling DQN 和 Prioritized Replay 都是在修正这些具体不稳定来源。

DQN 解决什么问题

Q-learning 的表格更新是：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

当状态空间巨大时，无法维护 Q-table。DQN 用神经网络近似 Q 函数：

$$Q(s, a; \theta)$$

如果动作空间是离散的，网络通常输入 s ，输出每个 action 的 Q 值：

$$\text{network}(s) \rightarrow [Q(s, a_1), Q(s, a_2), \dots, Q(s, a_n)]$$

DQN Loss

DQN 的 target：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

如果 s' 是终止状态：

$$y = r$$

损失函数：

$$L(\theta) = \mathbb{E}[(y - Q(s, a; \theta))^2]$$

其中：

- θ 是 online network 参数。
- θ' 是 target network 参数。

实际实现中常用 Huber loss：

$$L = \text{Huber}(y - Q(s, a; \theta))$$

Huber loss 对异常 TD error 更稳。

从 Q-learning 到 DQN Loss 的推理过程

Tabular Q-learning 的更新是：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [y - Q(s, a)]$$

其中：

$$y = r + \gamma \max_{a'} Q(s', a')$$

如果用神经网络 $Q(s, a; \theta)$ 代替表格，就不能直接“修改表格里的一個格子”。我们能做的是让网络当前输出靠近 target y 。

于是 RL 更新变成一个监督学习式回归问题：

```
input:  (s, a)
label:  y = r + gamma max Q_target(s', a')
predict: Q(s, a; theta)
loss:   (y - Q(s, a; theta))^2
```

这一步很关键：DQN 不是突然换了目标，它仍然在逼近 Bellman optimality backup，只是把 tabular update 改成了 neural network regression。

为什么 target 用 θ' 而不是当前 θ ？

- 如果 label 和 prediction 都由同一个快速变化的网络产生，训练目标会一直漂。
- target network 让 y 在一段时间内相对稳定。

- online network 只负责追这个相对固定的 Bellman target。

面试表达：

DQN 的 loss 可以看成把 Q-learning 的 TD target 当成监督信号。Replay buffer 解决样本相关性，target network 解决 bootstrap target 随参数同步漂移的问题。

DQN 训练流程

```
initialize online network Q(theta)
initialize target network Q(theta-) = Q(theta)
initialize replay buffer

for each step:
    choose action using epsilon-greedy
    execute action, observe r, s', done
    store (s, a, r, s', done) in replay buffer

    sample minibatch from replay buffer
    y = r + gamma (1 - done) max_a' Q(s', a'; theta-)
    minimize [y - Q(s, a; theta)]^2

    periodically update target network
```

Experience Replay

Replay buffer 存储 transition：

```
(s, a, r, s', done)
```

作用：

- 打破连续样本之间的相关性。
- 复用历史样本，提高 sample efficiency。
- 平滑训练数据分布。

面试表达：

DQN 中连续交互得到的样本高度相关，不适合直接做 SGD。Replay buffer 通过随机采样历史 transition，降低样本相关性，同时让每条经验被多次利用。

Target Network

如果 target 使用 online network：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

那么 θ 每次更新都会改变 target，训练会追逐一个移动目标。

DQN 使用 target network：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta)$$

θ 每隔固定步数从 θ 同步一次，或使用软更新。

面试表达：

Target network 让 bootstrap target 在一段时间内保持稳定，降低训练目标和预测网络同步变化带来的震荡。

Double DQN

DQN 中的 target：

$$\max_{a'} Q(s', a'; \theta)$$

同一个 max 操作既选择动作又评估动作，容易产生 overestimation bias。

Double DQN 将动作选择和动作评估拆开：

$$\begin{aligned} a^* &= \operatorname{argmax}_{a'} Q(s', a'; \theta) \\ y &= r + \gamma Q(s', a^*; \theta) \end{aligned}$$

其中：

- online network 负责选择动作。
- target network 负责评估该动作。

这样可以缓解 Q 值过估计。

Double DQN 为什么能缓解过估计

DQN 的问题来自 max operator。假设每个动作的 Q 估计都有噪声：

$$\text{estimated } Q = \text{true } Q + \text{noise}$$

当我们取 $\max_a Q(s', a)$ 时，更容易选中“被正噪声高估”的动作。即使每个动作估计平均无偏，最大值也会偏高。

普通 DQN 同一个 target network 同时做两件事：

1. 选择哪个动作最大。
2. 评估这个动作的价值。

Double DQN 把这两步拆开：

$$a^* = \operatorname{argmax}_a Q(s', a'; \theta)$$

$$y = r + \gamma Q(s', a^*; \theta')$$

这样选择动作时的噪声和评估动作时的噪声不完全相同，正噪声被重复放大的程度下降，因此 overestimation bias 更小。

面试表达：

Double DQN 的核心是 decouple action selection and action evaluation。DQN 的 max operator 容易选择被噪声高估的动作，Double DQN 用 online network 选动作，用 target network 估值，从而降低 overestimation bias。

Dueling DQN

Dueling DQN 将 Q 函数分解为状态价值和动作优势：

$$Q(s, a) = V(s) + A(s, a)$$

实际为了可辨识性，常用：

$$Q(s, a) = V(s) + A(s, a) - 1 / |\mathcal{A}| \sum_{a'} A(s, a')$$

直觉：

有些状态下，知道状态本身好不好比区分每个动作更重要。Dueling 结构让网络分别学习：

- $V(s)$ ：这个状态整体价值如何。
- $A(s, a)$ ：某个动作相对其他动作好多少。

适合动作之间差异不总是明显的场景。

Prioritized Experience Replay

普通 replay buffer 均匀采样 transition。

Prioritized Replay 更倾向采样 TD error 大的样本。

优先级：

$$p_i = |\delta_i| + \varepsilon$$

采样概率：

$$P(i) = p_i^\alpha / \sum_k p_k^\alpha$$

为了修正非均匀采样带来的 bias，使用 importance sampling weight：

$$w_i = (1 / (N P(i)))^\beta$$

直觉：

TD error 大的样本说明当前模型学得不好，更值得训练。

Rainbow DQN

Rainbow DQN 组合了多个 DQN 改进：

- Double DQN。
- Dueling Network。
- Prioritized Replay。
- Multi-step Learning。
- Distributional RL。
- NoisyNet。

面试中通常了解组件即可，不一定要求完整推导。

DQN 的适用场景

适合：

- 离散动作空间。
- 可以通过 Q 值枚举动作。
- 游戏、简单控制、离散决策。

不适合：

- 高维连续动作空间。
- 需要直接输出连续控制量的任务。
- 极端稀疏奖励且探索困难的任务。

连续控制通常用 DDPG、TD3、SAC、PPO 等算法。

本章例子：从 CartPole 到 DQN

例子 1：CartPole

CartPole 的状态是：

状态维度	含义
cart position	小车位置
cart velocity	小车速度
pole angle	杆角度
pole angular velocity	杆角速度

动作只有两个：

向左推 / 向右推

这类任务可以用 DQN 学：

输入状态 s

输出 $Q(s, \text{left}), Q(s, \text{right})$

选择 Q 最大的动作

例子 2：为什么需要 replay buffer

如果 agent 连续玩游戏，连续样本高度相关：

第 t 帧：球在左边

第 $t+1$ 帧：球稍微往右

第 $t+2$ 帧：球继续往右

直接按时间顺序训练，网络容易被最近局部轨迹带偏。Replay buffer 随机采样历史 transition，让 mini-batch 更接近独立同分布。

例子 3：Double DQN 过估计小例子

假设真实下一状态两个动作价值都接近 10，但网络估计有噪声：

$$Q(s', a1) = 10.1$$

$$Q(s', a2) = 9.8$$

普通 DQN 用同一个网络选动作和估值，会倾向选被噪声高估的动作。Double DQN 把“选动作”和“估值”拆开，可以降低这种系统性过估计。

面试表达：

DQN 的关键不是“神经网络版 Q-learning”这么简单，而是 replay buffer 和 target network 解决了深度网络训练时的相关样本和移动目标问题。

本章面试高频问题

1. DQN 相比 Q-learning 改了什么？

DQN 用神经网络近似 Q 函数，使 Q-learning 可以处理高维状态输入，例如图像。为了稳定训练，引入了 experience replay 和 target network。

2. DQN 为什么需要 replay buffer？

为了打破连续样本相关性，提高样本利用率，并使 mini-batch 训练更接近监督学习中的随机采样。

3. DQN 为什么需要 target network？

为了稳定 bootstrap target。如果 target 和 online prediction 使用同一网络，模型每次更新都会改变训练目标，容易震荡或发散。

4. Double DQN 解决什么问题？

解决 DQN 中 max operator 带来的 overestimation bias。它用 online network 选择动作，用 target network 评估动作，从而降低过估计。

5. Dueling DQN 的核心思想是什么？

将 Q 分解为状态价值 $V(s)$ 和动作优势 $A(s,a)$ ，让网络更好地学习状态本身价值和动作相对价值。

本章练习

1. 手写 DQN target 和 loss。
2. 解释 Double DQN 的 target 和 DQN target 的区别。
3. 画出 Dueling DQN 的网络结构。
4. 阅读一个 CleanRL DQN 实现，标出 replay buffer、target network、epsilon schedule 的位置。

第 7 章：Policy Gradient、REINFORCE（经典 Monte Carlo Policy Gradient 算法）与 Advantage

本章符号说明

符号/缩写	English	中文	含义
$\pi_\theta(a s)$	Parameterized Policy	参数化策略	参数为 θ 的策略分布。
θ	Policy Parameters	策略参数	policy network 的可学习参数。
τ	Trajectory	轨迹	一整段 state-action-reward 序列。
$J(\theta)$	Objective Function	目标函数	policy gradient 中要最大化的期望 return。
$R(\tau)$	Trajectory Return	轨迹回报	整条轨迹的累计 reward。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 开始的累计收益。
$b(s)$	Baseline	基线函数	用来降低 policy gradient 方差的 action-independent 函数。
$V_\pi(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_\pi(s,a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s,a) 的长期价值。
$A_\pi(s,a)$	Advantage Function	优势函数	$Q_\pi(s,a) - V_\pi(s)$ ，表示动作比平均水平好多少。
$H(\pi)$	Entropy	熵	衡量策略随机性，常用于鼓励探索。
RL	Reinforcement Learning	强化学习	Policy Gradient 是 RL 中直接优化策略的一类方法。
SAC	Soft Actor-Critic	软 Actor-Critic	后续最大熵 actor-critic 算法，本章先介绍 entropy 思想。

本章目标

本章学习 policy-based RL 的基础。你需要掌握：

- 为什么需要直接优化 policy。
- stochastic policy。
- policy gradient theorem。

- log-derivative trick。
- REINFORCE。
- baseline。
- advantage function。

本章主线

Value-based 方法先学 $Q(s,a)$ ，再通过 $\operatorname{argmax}_a Q(s,a)$ 选动作。这个思路在连续动作、随机策略或需要直接控制动作分布时不够自然。本章转向 policy-based RL：直接参数化 $\pi_\theta(a/s)$ ，推导如何对期望 return 求梯度，并用 baseline 和 advantage 降低方差。

为什么需要 Policy Gradient

Value-based 方法学习 $Q(s,a)$ ，再通过 $\operatorname{argmax}_a Q(s,a)$ 选动作。它在离散动作空间中很自然，但在以下场景中不够方便：

- 动作空间连续。
- 策略本身需要随机性。
- 希望直接优化策略分布。
- $\operatorname{argmax}_a Q(s,a)$ 很难计算。

Policy Gradient 直接参数化策略：

$$\pi_\theta(a \mid s)$$

目标是最大化期望 return：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

其中 trajectory：

$$\tau = (s_0, a_0, r_1, s_1, a_1, \dots)$$

随机策略

离散动作空间中，策略可以是 categorical distribution：

$$\pi_\theta(a \mid s) = \operatorname{softmax}(f_\theta(s))$$

连续动作空间中，策略常用 Gaussian：

$$\pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$$

随机策略的好处：

- 自带探索。
- 可以表达多模态或不确定行为。

- 适合 policy gradient 的概率形式推导。

Log-derivative Trick

核心恒等式：

$$\nabla_{\theta} p_{\theta}(x) = p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)$$

因为：

$$\nabla_{\theta} \log p_{\theta}(x) = \nabla_{\theta} p_{\theta}(x) / p_{\theta}(x)$$

这个技巧让我们可以对采样分布依赖参数的期望求梯度。

目标：

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)}[R(\tau)]$$

梯度：

$$\begin{aligned} \nabla J(\theta) &= \nabla \mathbb{E}_{\tau \sim p_{\theta}(\tau)} R(\tau) \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \nabla p_{\theta}(\tau) R(\tau) \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \nabla \log p_{\theta}(\tau) p_{\theta}(\tau) R(\tau) \\ &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla \log p_{\theta}(\tau) R(\tau)] \end{aligned}$$

trajectory 概率：

$$\begin{aligned} p_{\theta}(\tau) &= p(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t) \end{aligned}$$

环境转移 P 不依赖 θ ，所以：

$$\begin{aligned} \nabla \log p_{\theta}(\tau) &= \sum_t \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

得到 policy gradient：

$$\begin{aligned} \nabla J(\theta) &= \mathbb{E}_{\tau} [\sum_t \nabla \log \pi_{\theta}(a_t | s_t) R(\tau)] \end{aligned}$$

Policy Gradient 推导阅读线

这段推导最容易觉得“公式突然变形”。可以按下面逻辑理解。

第一步，目标函数是期望：

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau)]$$

难点在于：被求期望的轨迹分布 $p_{\theta}(\tau)$ 本身依赖策略参数 θ 。参数变了，采样出来的轨迹分布也变了。

第二步，把期望写成积分或求和：

$$J(\theta) = \mathbb{E}_{\tau} p_{\theta}(\tau) R(\tau) d\tau$$

由于 reward 函数本身通常不直接对策略参数求导，梯度主要作用在轨迹概率上：

$$\nabla J(\theta) = \mathbb{E}_{\tau} \nabla p_{\theta}(\tau) R(\tau) d\tau$$

第三步，用 log-derivative trick 把 ∇p 改写成 $p \nabla \log p$ ：

$$\nabla p_{\theta}(\tau) = p_{\theta}(\tau) \nabla \log p_{\theta}(\tau)$$

这样积分又变回期望：

$$\nabla J(\theta) = \mathbb{E}_{\tau} [\nabla \log p_{\theta}(\tau) R(\tau)]$$

第四步，展开轨迹概率：

$$p_{\theta}(\tau) = p(s_0) \prod_t \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

这里环境初始分布和转移概率不由策略参数控制，所以对 θ 求梯度时只剩策略项：

$$\nabla \log p_{\theta}(\tau) = \sum_t \nabla \log \pi_{\theta}(a_t | s_t)$$

最后得到：

$$\nabla J(\theta) = \mathbb{E}_{\tau} [\sum_t \nabla \log \pi_{\theta}(a_t | s_t) R(\tau)]$$

一句话记忆：

因为采样动作本身不可直接反传，所以 policy gradient 不对 action 求导，而是对“采到这个 action 的 log probability”求导。高 return 的轨迹会提高其中动作的 log probability，低 return 的轨迹会降低它们。

REINFORCE

REINFORCE 是最基础的 Monte Carlo policy gradient 算法。

更新方向：

$$\theta \leftarrow \theta + \alpha \nabla \log \pi_{\theta}(a_t | s_t) G_t$$

直觉：

- 如果某个 action 后续 return 高，就增加该 action 在该 state 下的概率。
- 如果 return 低，就降低该 action 的概率。

算法流程：

```
initialize policy pi_theta
repeat:
    sample trajectories using pi_theta
    compute returns G_t
    update theta by sum_t grad log pi_theta(a_t|s_t) G_t
```

REINFORCE 的优点：

- 概念简单。
- 不需要 value function。
- 可以直接处理随机策略。

缺点：

- 必须等 episode 结束。
- Monte Carlo return 方差很大。
- sample efficiency 较低。

为什么 REINFORCE 可以用 G_t

基础推导中，每个时间步都乘整条轨迹回报 $R(\tau)$ 。但动作 a_t 不可能影响 t 之前已经发生的 reward，所以更合理的学习信号是从当前时间开始的 reward-to-go：

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots$$

于是实际常用：

$$\nabla J(\theta) = \mathbb{E}[\sum_t \nabla \log \pi_\theta(a_t | s_t) G_t]$$

这样做不会丢掉由 a_t 影响的未来回报，反而去掉了和 a_t 无关的过去 reward，因此方差更低。

Baseline

为了降低方差，可以从 return 中减去 baseline：

$$\begin{aligned} \nabla J(\theta) \\ = \mathbb{E}[\nabla \log \pi_\theta(a_t | s_t) (G_t - b(s_t))] \end{aligned}$$

常用 baseline：

$$b(s_t) = V(s_t)$$

为什么 baseline 不引入 bias?

因为：

$$\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a|s) b(s)] = 0$$

推导：

$$\begin{aligned} & \sum_a \pi(a|s) \nabla \log \pi(a|s) b(s) \\ &= b(s) \sum_a \pi(a|s) \nabla \log \pi(a|s) \\ &= b(s) \sum_a \nabla \pi(a|s) \\ &= b(s) \nabla \sum_a \pi(a|s) \\ &= b(s) \nabla 1 \\ &= 0 \end{aligned}$$

只要 baseline 不依赖 action，就不会改变 policy gradient 的期望，只会改变方差。

更直观地说，baseline 只是把“绝对分数”改成“相对分数”：

- 原来：这次 return 是 100，就增加动作概率。
- 加 baseline 后：如果这个状态平均就能拿 120，那么 100 其实低于预期，应该降低动作概率。

这就是 advantage 的来源。

Advantage Function

Advantage 定义：

$$A_{\pi}(s, a) = Q_{\pi}(s, a) - V_{\pi}(s)$$

含义：

在状态 s 下，动作 a 比当前策略的平均动作好多少。

使用 advantage 的 policy gradient：

$$\begin{aligned} & \nabla J(\theta) \\ &= \mathbb{E}[\nabla \log \pi_{\theta}(a_t | s_t) A_{\pi}(s_t, a_t)] \end{aligned}$$

直觉：

- 如果 $A(s, a) > 0$ ，说明动作比平均水平好，增加其概率。
- 如果 $A(s, a) < 0$ ，说明动作比平均水平差，降低其概率。

相比直接用 return，advantage 通常方差更低，学习信号更明确。

从 Baseline 到 Actor-Critic

如果 baseline 取 $V_{\pi}(s)$ ，那么：

$$G_t - V_{\pi}(s_t)$$

就是一个 advantage 的 Monte Carlo 估计。问题是 G_t 仍然要等 episode 结束且方差大。

Actor-Critic 的下一步就是：训练一个 critic 来估计 $V(s)$ 或 $Q(s,a)$ ，用 TD target 更快、更低方差地构造 advantage。也就是说：

REINFORCE: 用完整 return 做策略更新
 REINFORCE+b: 用 return - baseline 降低方差
 Actor-Critic: 用 critic 估计 baseline / advantage
 GAE/PP0: 用更稳定的 advantage 和受限策略更新

所以第 7 章和第 8/9 章不是割裂的：后面的 actor-critic、GAE、PPO 都是在让 policy gradient 的学习信号更稳。

Reward-to-go

原始 REINFORCE 可能用整条轨迹 return $R(\tau)$ 更新每个时间步。
 更合理的是使用从当前时间步开始的 return：

$$G_t = \sum_{k=t}^T \gamma^{k-t} R_{k+1}$$

原因：

时间 t 的 action 不会影响过去已经发生的 reward，因此不需要用过去 reward 作为学习信号。

Entropy Regularization

为了鼓励探索，常在目标中加入 entropy：

$$J(\theta) = \mathbb{E}[\text{return}] + \beta \mathbb{E}[H(\pi_{\theta}(\cdot | s))]$$

策略熵：

$$H(\pi(\cdot | s)) = - \sum_a \pi(a | s) \log \pi(a | s)$$

作用：

- 防止策略过早变得确定。
- 增强探索。
- 改善训练稳定性。

SAC 中 entropy regularization 是核心组成部分。

本章例子：为什么需要直接优化策略

例子 1：连续动作机器人

机器人手臂要输出关节力矩：

```
a = [torque_1, torque_2, torque_3]
```

如果用 DQN，要对连续动作求 $\max_a Q(s,a)$ ，这个优化很难。Policy Gradient 直接学习：

$$\pi_{\theta}(a|s)$$

或确定性策略：

$$a = \mu_{\theta}(s)$$

这样动作可以直接由策略网络输出。

例子 2：Softmax 策略更新

假设在某个状态有两个动作：

动作	当前概率	采样后 return
left	0.4	+5
right	0.6	-1

Policy Gradient 会提高产生高 return 的动作概率，降低低 return 动作概率。

直觉：

```
采样到 left 且结果好 -> 增大 log pi(left|s)
采样到 right 且结果差 -> 减小 log pi(right|s)
```

例子 3：Baseline 的作用

如果一个游戏平均每局 return 是 100：

- 某次动作后 return 是 105，不算特别好，只比平均好 5。
- 另一动作后 return 是 20，明显差。

使用 baseline 后，更新看的是：

$$G_t - V(s)$$

这比直接用 G_t 更能判断动作相对好坏，方差更低。

面试表达：

Policy Gradient 的核心例子是连续控制和随机策略。它不需要枚举动作，也不需要为 Q 做 $\arg\max$ ，而是让高回报轨迹上的动作概率上升。

本章面试高频问题

1. Policy Gradient 适合什么场景？

适合连续动作空间、随机策略建模、需要直接优化策略分布的场景。相比 value-based 方法，不需要对动作做 $\arg\max$ 。

2. REINFORCE 的核心更新是什么？

$$\theta \leftarrow \theta + \alpha \nabla \log \pi_{\theta}(a_t | s_t) G_t$$

如果某个动作带来高 return，就提升该动作概率；如果 return 低，就降低该动作概率。

3. 为什么 baseline 不改变梯度期望？

因为 baseline 不依赖 action，而 $\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a|s)] = 0$ 。所以减去 baseline 不引入 bias，只降低 variance。

4. Advantage 的意义是什么？

Advantage 衡量某个动作相对于该状态平均动作的好坏。它比直接用 return 更能反映动作的相对贡献，通常降低方差并稳定训练。

5. Policy Gradient 的主要缺点？

方差较大、sample efficiency 较低、训练对学习率敏感。REINFORCE 尤其需要完整 episode，训练慢且不稳定。

本章练习

1. 手写 log-derivative trick 推导。
2. 证明 action-independent baseline 不引入 bias。
3. 解释 $Q(s,a)$ 、 $V(s)$ 、 $A(s,a)$ 的关系。
4. 对比 value-based 和 policy-based 方法。

第 8 章：Actor-Critic、A2C (Advantage Actor-Critic, 优势 Actor-Critic)、A3C (Asynchronous Advantage Actor-Critic, 异步优势 Actor-Critic) 与 GAE (Generalized Advantage Estimation, 广义优势估计)

本章符号说明

符号/缩写	English	中文	含义
A2C	Advantage Actor-Critic	优势 Actor-Critic	同步采样和更新的 actor-critic 方法。
A3C	Asynchronous Advantage Actor-Critic	异步优势 Actor-Critic	多 worker 异步更新的 actor-critic 方法。
GAE	Generalized Advantage Estimation	广义优势估计	用 λ 在 TD 和 MC advantage 之间折中。
$\pi_{\theta}(a s)$	Actor Policy	Actor 策略	actor 网络输出的动作分布。
$V_w(s)$	Critic Value Function	Critic 状态价值函数	critic 网络对状态价值的估计。
$Q_w(s, a)$	Critic Action-value Function	Critic 动作价值函数	critic 网络对状态-动作价值的估计。
$J(\theta)$	Policy Objective	策略目标函数	actor 要最大化的目标。
δ_t	TD Error	时序差分误差	可作为一步 advantage 估计。
A_t	Advantage Estimate	优势估计	策略更新时衡量动作好坏的信号。
L_{policy} / L_{value}	Policy Loss / Value Loss	策略损失 / 价值损失	actor 和 critic 的训练损失。
$R_t^{(n)}$	n-step Return	n 步回报	n-step bootstrap target。
λ	GAE Lambda	GAE 衰减参数	控制 bias 和 variance 的折中。
TD	Temporal Difference	时序差分	critic 常用 TD error 做估计。
MC	Monte Carlo	蒙特卡洛	GAE 的另一端近似完整 return。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	Actor-Critic 要改进的高方差 policy gradient 基线。

符号/缩写	English	中文	含义
PPO	Proximal Policy Optimization	近端策略优化	常结合 GAE 使用的策略优化算法。

本章目标

本章学习 Actor-Critic 框架。你需要掌握：

- actor 和 critic 的职责。
- Actor-Critic 如何结合 policy gradient 和 value learning。
- TD error 如何作为 advantage 估计。
- A2C 和 A3C。
- Generalized Advantage Estimation。

本章主线

第 7 章的纯 policy gradient 可以直接优化策略，但 Monte Carlo return 方差高。第 3 章的 TD value learning 方差低但主要解决估值。本章把两者合并：actor 负责更新策略，critic 负责估计 value 或 advantage。GAE 则进一步在 MC 和 TD 之间调节 bias/variance。

Actor-Critic 基本思想

Actor-Critic 有两个部分：

- Actor：策略网络 $\pi_{\theta}(a|s)$ ，负责选动作。
- Critic：价值网络 $V_w(s)$ 或 $Q_w(s,a)$ ，负责评估动作或状态。

Actor 用 critic 提供的学习信号更新策略：

$$\begin{aligned} \nabla_{\theta} J(\theta) \\ = \mathbb{E}[\nabla \log \pi_{\theta}(a|s) A(s,a)] \end{aligned}$$

Critic 学习价值函数：

$$V_w(s) \approx V_{\pi}(s)$$

直觉：

Actor 负责做决策，critic 负责评价决策质量。Critic 降低了纯 Monte Carlo policy gradient 的方差，让策略更新更稳定。

TD Error 作为 Advantage

TD error：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

它可以看成一步 advantage 的估计：

$$A(s_t, a_t) \approx \delta_t$$

直觉：

- 如果实际 reward 加下一状态价值高于当前价值估计，说明这个动作比预期好。
- 如果低于当前价值估计，说明这个动作比预期差。

Actor 更新：

$$\theta \leftarrow \theta + \alpha \nabla \log \pi_{\theta}(a_t | s_t) \delta_t$$

Critic 更新：

$$w \leftarrow w - \beta \nabla (r_t + \gamma V_w(s_{t+1}) - V_w(s_t))^2$$

为什么 TD Error 可以近似 Advantage

Advantage 的定义是：

$$A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$$

而动作价值可以拆成“一步 reward + 下一状态价值”：

$$Q_{\pi}(s_t, a_t) = \mathbb{E}[r_t + \gamma V_{\pi}(s_{t+1})]$$

把它代入 advantage：

$$A_{\pi}(s_t, a_t) = \mathbb{E}[r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)]$$

如果用一次采样近似这个期望，就得到：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

所以 TD error 不是随便定义的误差，它正是“一步采样下的 advantage 估计”。

面试里可以这样讲：

如果执行动作后得到的 reward 加下一状态价值超过当前状态价值，说明这个动作比平均动作好，advantage 为正；反之说明动作低于预期，advantage 为负。

Actor-Critic vs REINFORCE

维度	REINFORCE	Actor-Critic
策略更新信号	Monte Carlo return	critic 估计的 advantage
是否 bootstrap	否	通常是
方差	高	较低
bias	较低	有 critic bias
是否能在线更新	通常需要 episode 结束	可以在线或 n-step 更新
sample efficiency	较低	更高

面试表达：

Actor-Critic 用 critic 估计 value 或 advantage，降低 policy gradient 的方差，并允许 TD 式在线更新。但 critic 是函数逼近模型，因此会引入估计 bias。

A2C

A2C 是 Advantage Actor-Critic。

目标通常包含三部分：

1. Policy loss。
2. Value loss。
3. Entropy bonus。

Policy loss：

$$L_{policy} = - \log \pi_{\theta}(a_t | s_t) A_t$$

Value loss：

$$L_{value} = (R_t - V_w(s_t))^2$$

Entropy bonus：

$$L_{entropy} = - \beta H(\pi_{\theta}(\cdot | s_t))$$

整体最小化：

$$L = L_{policy} + c_v L_{value} + L_{entropy}$$

注意符号：很多实现中是最小化 loss，因此 policy gradient 的最大化目标会写成负号。

A3C

A3C 是 Asynchronous Advantage Actor-Critic。

核心思想：

- 多个 worker 并行和环境交互。
- 每个 worker 异步更新共享全局网络。
- 不同 worker 产生的数据降低样本相关性。

A3C 在 replay buffer 不常用的 on-policy 场景中，通过并行采样改善数据多样性。

A2C 可以看作 A3C 的同步版本：

- A3C：asynchronous。
- A2C：synchronous，多个环境同步采样后统一更新。

N-step Return

一步 TD target：

$$R_t^{(1)} = r_t + \gamma V(s_{t+1})$$

n-step return：

$$R_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$$

n-step return 在 MC 和 TD 之间折中：

- n 小：bias 较大，variance 较小。
- n 大：bias 较小，variance 较大。

Generalized Advantage Estimation

GAE 用一个参数 λ 在不同步数的 advantage 估计之间折中。

TD residual：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE：

$$A_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

直觉：

- $\lambda = 0$ ：接近一步 TD，bias 大、variance 小。
- $\lambda = 1$ ：接近 Monte Carlo，bias 小、variance 大。
- 实践中常用 $\lambda = 0.95$ 。

PPO 中经常使用 GAE 估计 advantage。

GAE 的推理过程

一步 TD error 方差小，但依赖 critic，bias 可能大。完整 Monte Carlo return bias 小，但方差大。GAE 的想法是：不要只选一个步数，而是把不同步数的 TD residual 加权平均。

从一步 TD residual 开始：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

如果下一步也持续“比预期好”，那么 δ_{t+1} 也应该影响当前动作的评价，只是影响要打折：

$$\delta_t + \gamma \lambda \delta_{t+1}$$

继续往后累加，就得到：

$$A_t^{GAE} = \delta_t + \gamma \lambda \delta_{t+1} + (\gamma \lambda)^2 \delta_{t+2} + \dots$$

也就是：

$$A_t^{GAE} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

这里 γ 是时间折扣， λ 是“愿意看多远的误差信号”：

- λ 小：更相信短期 bootstrap，更新稳但 bias 大。
- λ 大：更接近长回报，bias 小但 variance 大。

实践里 PPO 常用 $\lambda=0.95$ ，是因为它通常在稳定性和学习信号之间比较均衡。

Shared Network vs Separate Network

Actor 和 critic 可以：

- 共享一部分 backbone，分别输出 policy head 和 value head。
- 完全分离成两个网络。

共享 backbone 的优点：

- 参数更少。
- 表征可以复用。

风险：

- policy loss 和 value loss 的梯度可能相互干扰。
- 需要调节 value loss coefficient。

本章例子：Actor 和 Critic 分工

例子 1：游戏 agent

在一个游戏中：

- Actor 负责选择动作，例如移动、攻击、防御。
- Critic 负责评价当前局面值多少钱。

如果 Actor 做了一个动作后，Critic 发现：

$$r + \gamma V(s') - V(s) > 0$$

说明这步结果比预期好，Actor 应该增加这个动作在该状态下的概率。

如果 TD error 小于 0，则说明这步比预期差，Actor 应该降低该动作概率。

例子 2：A2C 和 REINFORCE 对比

REINFORCE 要等一整局结束，用完整 G_t 更新策略。A2C 用 critic 提供一步或多步 target，可以更早更新。

REINFORCE target: G_t
A2C target: $r + \gamma V(s')$

所以 A2C 通常方差更低，但 critic 不准时会带来 bias。

例子 3：GAE 的直觉

如果只用一步 TD error，更新稳定但可能短视。

如果用完整 return，信息完整但方差大。

GAE 用 λ 在中间插值：

- λ 接近 0：更像一步 TD。
- λ 接近 1：更像 Monte Carlo。

面试表达：

Actor-Critic 可以理解成 policy gradient 加一个会估值的助手。Critic 不直接决定动作，而是告诉 Actor 这次动作比预期好还是差。

本章面试高频问题

1. Actor-Critic 中 actor 和 critic 分别做什么？

Actor 学习策略并负责选动作。Critic 学习价值函数，用来评价 actor 的动作，并为 policy gradient 提供低方差的 advantage 估计。

2. Actor-Critic 相比 REINFORCE 的优势？

Actor-Critic 用 critic 的 value estimate 替代完整 Monte Carlo return，降低方差，并支持在线或 n-step 更新，sample efficiency 更高。

3. TD error 为什么可以作为 advantage?

TD error 衡量执行动作后的一步结果是否比当前价值估计更好。如果 $r + \gamma V(s') > V(s)$, 说明动作带来的结果超出预期, 可以增加其概率。

4. A2C 和 A3C 的区别?

A3C 使用多个 worker 异步更新全局网络。A2C 是同步版本, 多个环境同步采样后统一更新, 工程上更稳定、易并行。

5. GAE 解决什么问题?

GAE 在 bias 和 variance 之间做平滑折中, 通过 λ 控制 advantage 估计更接近一步 TD 还是 Monte Carlo return。

本章练习

1. 写出 Actor-Critic 的 actor loss 和 critic loss。
2. 解释 TD error 和 advantage 的关系。
3. 推导 n-step return。
4. 说明 GAE 中 λ 的作用。

第 9 章：TRPO (Trust Region Policy Optimization, 信赖域策略优化)、PPO (Proximal Policy Optimization, 近端策略优化) 与策略更新约束

本章符号说明

符号/缩写	English	中文	含义
TRPO	Trust Region Policy Optimization	信赖域策略优化	用 KL 约束限制策略更新幅度。
PPO	Proximal Policy Optimization	近端策略优化	用 clipping objective 近似限制策略更新幅度。
KL	Kullback-Leibler Divergence	KL 散度	衡量新旧策略分布差异。
θ / θ_{old}	New / Old Policy Parameters	新 / 旧策略参数	当前要优化的策略参数和采样数据时的旧参数。
$r_t(\theta)$	Probability Ratio	概率比	新策略和旧策略对同一动作的概率比。
A_t	Advantage Estimate	优势估计	判断当前动作应提高还是降低概率的信号。
$L^{CLIP}(\theta)$	Clipped Objective	截断目标函数	PPO 用来限制策略更新的 surrogate objective。
ϵ	Clipping Range	截断范围	PPO 中控制 ratio 允许变化范围的超参数。
L_{policy} / L_{value}	Policy Loss / Value Loss	策略损失 / 价值损失	PPO 总 loss 的两个主要部分。
$H(\pi)$	Entropy	熵	鼓励策略保持探索的正则项。
CLIP	Clipped Objective Label	截断目标标记	L^{CLIP} 中的上标, 表示 PPO 的 clipped surrogate objective。
GAE	Generalized Advantage Estimation	广义优势估计	PPO 中常用的 advantage 估计方法。
DQN	Deep Q-Network	深度 Q 网络	PPO 对比中的 value-based deep RL 算法。

符号/缩写	English	中文	含义
RL / RLHF	Reinforcement Learning / Reinforcement Learning from Human Feedback	强化学习 / 基于人类反馈的强化学习	PPO 既用于 RL，也常出现在 RLHF 流程中。

本章目标

本章学习现代 on-policy deep RL 中最常用的一类算法。你需要掌握：

- 为什么 policy gradient 需要限制更新步长。
- importance sampling ratio。
- TRPO 的 trust region 思想。
- PPO clipping objective。
- PPO 的训练细节。
- PPO 的优缺点。

本章主线

Actor-Critic 解决了 policy gradient 高方差问题，但策略更新仍可能一步走太大，导致新策略和采样数据对应的旧策略差异过大。本章专门处理“更新幅度控制”：TRPO 用 trust region 和 KL 约束，PPO 用 clipping objective 做更易实现的近似。

为什么要限制策略更新

Policy Gradient 直接更新策略参数：

$$\theta \leftarrow \theta + \alpha \nabla J(\theta)$$

但策略优化很敏感。如果一次更新太大，新的 policy 和采样数据对应的旧 policy 差异过大，可能导致性能崩掉。

原因：

- on-policy 数据只对采样它的旧策略可靠。
- 策略变化太大时，旧数据不能准确估计新策略表现。
- neural policy 的参数小变化可能导致动作分布大变化。

因此 TRPO/PPO 都试图限制新旧策略距离。

Importance Sampling Ratio

PPO/TRPO 常用 ratio：

$$r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$$

它表示在同一个 (s_t, a_t) 上，新策略相对于旧策略的概率变化。

Policy objective 可写成：

$$L(\theta) = \mathbb{E}[r_t(\theta) A_t]$$

如果 $A_t > 0$ ，希望增加该动作概率，即增大 ratio。

如果 $A_t < 0$ ，希望降低该动作概率，即减小 ratio。

Ratio 从哪里来

PPO 的数据是旧策略 $\pi_{\theta_{old}}$ 采样出来的，但我们想评估新策略 π_{θ} 。如果完全重新采样，成本很高；如果直接把旧数据当新数据，又会有分布偏差。

importance sampling 的基本想法是：

$$\mathbb{E}_{a \sim \pi_{\theta}}[f(a)] = \mathbb{E}_{a \sim \pi_{old}}[\pi_{\theta}(a|s) / \pi_{old}(a|s) f(a)]$$

所以出现 ratio：

$$r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t)$$

它把“旧策略采到的动作”校正成“新策略下这个动作有多可能”。但 ratio 如果太大或太小，说明新旧策略差异太大，旧数据对新策略的估计就不可靠。这就是后面 clipping 的动机。

TRPO

TRPO 是 Trust Region Policy Optimization。

它希望在提升 surrogate objective 的同时，限制新旧策略的 KL divergence：

$$\begin{aligned} & \max_{\theta} \mathbb{E}[r_t(\theta) A_t] \\ & \text{subject to } \mathbb{E}[KL(\pi_{old}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t))] \leq \delta \end{aligned}$$

核心思想：

每次策略更新不能离旧策略太远，保证更新在 trust region 内。

优点：

- 理论上更稳定。
- 避免灾难性大步更新。

缺点：

- 实现复杂。
- 需要二阶近似、conjugate gradient、line search。
- 工程成本较高。

PPO

PPO 是 Proximal Policy Optimization。它保留了限制策略更新幅度的思想，但实现更简单。

PPO clipped objective:

$$\begin{aligned} L^{CLIP}(\theta) \\ &= \mathbb{E}[\min(r_t(\theta) A_t, \\ &\quad \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] \end{aligned}$$

其中:

$$r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{old}}(a_t | s_t)$$

ϵ 常用 0.1 或 0.2。

PPO Clipping 直觉

当 $A_t > 0$:

- 增大动作概率是好事。
- 但 ratio 超过 $1 + \epsilon$ 后，收益被截断。
- 防止过度增加该动作概率。

当 $A_t < 0$:

- 降低动作概率是好事。
- 但 ratio 低于 $1 - \epsilon$ 后，收益被截断。
- 防止过度降低该动作概率。

一句话:

PPO clipping 允许策略朝 advantage 指示的方向更新，但限制更新幅度，避免新旧策略差异过大。

Clipping 分情况推理

PPO 的 min 不是为了让公式复杂，而是为了“好方向可以走，但不要走太猛”。

情况 1: $A_t > 0$

动作比平均水平好，应该提高它的概率。未截断目标是:

$$r_t A_t$$

当 r_t 从 1 增大时，目标变大，表示鼓励新策略更常选这个动作。但如果 $r_t > 1 + \epsilon$ ，说明概率已经提高太多。此时 clipped 项固定在:

$$(1+\epsilon)A_t$$

再提高概率也不会得到更多收益，梯度被压住。

情况 2: $A_t < 0$

动作比平均水平差，应该降低它的概率。因为 A_t 是负数，减小 r_t 会让 $r_t A_t$ 变大，看起来是好事。但如果 $r_t < 1-\epsilon$ ，说明概率已经降低太多。clipped 项固定在：

$$(1-\epsilon)A_t$$

PPO 取 min，会阻止继续通过过度降低概率来“刷目标”。

因此 clipped objective 同时限制两种过度更新：

- 好动作概率不能一下子加太多。
- 坏动作概率不能一下子降太多。

这就是 PPO 比普通 policy gradient 更稳的核心。

PPO Loss 组成

常见 PPO 总 loss：

$$L = L_{policy} + c_v L_{value} - c_e Entropy$$

如果按最小化写：

$$L_{policy} = - \mathbb{E}[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

Value loss：

$$L_{value} = \mathbb{E}[(V(s_t) - R_t)^2]$$

Entropy bonus：

$$Entropy = \mathbb{E}[H(\pi(\cdot | s_t))]$$

实际实现中还可能：

- advantage normalization。
- value clipping。
- gradient clipping。
- target KL early stopping。
- learning rate annealing。

PPO 训练流程

```

for each iteration:
    collect rollout using pi_old
    compute returns and advantages, often with GAE
    for several epochs:
        shuffle rollout data
        split into minibatches
        update policy with clipped objective
        update value function
    set pi_old <- current policy

```

注意：

PPO 是 on-policy，但它会对同一批 rollout 做多个 epoch 的 minibatch 更新。clipping 机制就是为了让这种有限复用不至于让策略偏离太远。

PPO 一轮训练的推理顺序

从实现角度，PPO 一轮迭代可以按下面理解：

1. 用当前策略采样 rollout，此时这份数据绑定的是 π_{old} 。
2. 用 reward 和 critic 计算 return。
3. 用 GAE 计算 advantage，得到每个动作“比平均水平好多少”。
4. 保存旧策略对已采样动作的 log probability。
5. 更新时重新计算新策略 log probability。
6. 用新旧 log probability 的差得到 ratio。
7. 用 clipped objective 更新 actor。
8. 用 return target 更新 critic。
9. 用 entropy bonus 防止策略过早塌缩。

这里最关键的是第 4 到第 7 步：PPO 不是只看当前策略输出，而是一直比较“新策略相对采样时旧策略改了多少”。

PPO 为什么常用

优点：

- 实现比 TRPO 简单。
- 稳定性好。
- 对超参数相对不那么敏感。
- 离散和连续动作都能用。
- 成为很多 RLHF 早期流程中的常见选择。

缺点：

- on-policy，sample efficiency 不如 off-policy 方法。
- 需要大量环境交互。
- 对 reward scaling、advantage normalization 仍然敏感。

- clipping 不是严格 trust region，仍可能出现不稳定。

PPO vs DQN

维度	DQN	PPO
类型	value-based	policy-based / actor-critic
动作空间	离散	离散和连续
策略	由 Q argmax 导出	直接学习 policy
数据使用	off-policy	on-policy
样本效率	较高	较低
稳定性	需要 replay 和 target network	clipping + GAE 较稳定

本章例子：PPO clipping 怎么工作

假设某个动作在旧策略下概率是 0.20，新策略下概率是 0.26。

importance ratio 是：

$$r_t = 0.26 / 0.20 = 1.3$$

如果 $\epsilon=0.2$ ，允许范围是：

$$[0.8, 1.2]$$

Advantage 为正

如果 $A_t > 0$ ，说明这个动作比预期好，应该提高概率。但 ratio 已经到 1.3，超过 1.2，PPO 会把收益截断在 1.2，防止策略一步变太多。

Advantage 为负

如果 $A_t < 0$ ，说明这个动作不好，应该降低概率。但如果新策略把概率降得过猛，ratio 低于 0.8，也会被 clip。

RLHF 例子

在 RLHF 里，策略是语言模型，动作可以理解成生成 token。PPO 更新时：

- reward model 给回答打分。
- critic 估计 value。
- advantage 判断这段输出比预期好还是差。
- KL 或 clipping 防止模型偏离参考模型太远。

面试表达：

PPO 的例子要讲清楚 ratio。它不是禁止策略更新，而是限制同一批数据上新旧策略概率比变化过大，避免 policy gradient 一步把策略推崩。

本章面试高频问题

1. PPO 解决了什么问题？

PPO 解决普通 policy gradient 更新过大导致策略崩掉的问题。它通过 clipping objective 限制新旧策略概率比的变化，使策略更新更稳定。

2. PPO 的 ratio 是什么？

$$r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{old}(a_t | s_t)$$

它衡量新策略相对于采样数据的旧策略，对同一动作概率改变了多少。

3. PPO clipping objective 的直觉？

如果 advantage 为正，允许提高动作概率，但超过 $1 + \epsilon$ 后不再给额外收益。如果 advantage 为负，允许降低动作概率，但低于 $1 - \epsilon$ 后不再给额外收益。

4. PPO 和 TRPO 的区别？

TRPO 通过 KL constraint 显式限制策略更新，理论更严谨但实现复杂。PPO 用 clipping 或 KL penalty 近似 trust region 思想，实现简单，工程上更常用。

5. PPO 是 on-policy 还是 off-policy？

PPO 通常是 on-policy。虽然它会对同一批 rollout 做多个 epoch 更新，但数据仍来自当前旧策略，且通过 ratio 和 clipping 控制偏移。

本章练习

1. 手写 PPO clipped objective。
2. 分别解释 $A_t > 0$ 和 $A_t < 0$ 时 clipping 的效果。
3. 对比 TRPO 和 PPO。
4. 说明 PPO 中 GAE、entropy bonus、advantage normalization 的作用。

第 10 章：连续控制、DDPG (Deep Deterministic Policy Gradient, 深度确定性策略梯度)、TD3 (Twin Delayed DDPG, 双延迟 DDPG) 与 SAC (Soft Actor-Critic, 软 Actor-Critic)

本章符号说明

符号/缩写	English	中文	含义
DDPG	Deep Deterministic Policy Gradient	深度确定性策略梯度	连续动作空间的 off-policy actor-critic 算法。
TD3	Twin Delayed DDPG	双延迟 DDPG	用双 Q、延迟 actor 更新和 target smoothing 改进 DDPG。
SAC	Soft Actor-Critic	软 Actor-Critic	最大熵 off-policy actor-critic 算法。
$Q_w(s, a)$	Critic Action-value Function	Critic 动作价值函数	参数 w 下的 Q 估计。
$\mu_\theta(s)$	Deterministic Actor	确定性 actor	DDPG/TD3 中 actor 输出的连续动作。
$\pi_\theta(a s)$	Stochastic Actor	随机 actor	SAC 中 actor 输出的动作分布。
y	Bellman Target	Bellman 目标	critic 训练时的目标值。
$J(\theta)$	Actor Objective	Actor 目标函数	actor 要最大化或等价最小化的目标。
Q_1 / Q_2	Twin Q Networks	双 Q 网络	TD3/SAC 中用于缓解过估计的两个 critic。
$H(\pi)$	Entropy	熵	SAC 目标中鼓励策略随机性的项。
α	Temperature	温度系数	SAC 中 reward 和 entropy 的权衡参数。
DQN	Deep Q-Network	深度 Q 网络	离散动作空间的 value-based 算法，本章用它引出连续动作困难。

本章目标

本章学习连续动作空间中的经典 off-policy actor-critic 算法。你需要掌握：

- 连续动作为什么不适合普通 DQN。
- DDPG。
- TD3。
- SAC。
- entropy regularization。
- deterministic policy vs stochastic policy。

本章主线

第 6 章的 DQN 依赖离散动作枚举，第 9 章的 PPO 是 on-policy，样本效率相对低。连续控制通常既需要 actor 直接输出连续动作，又希望 off-policy 复用 replay buffer。本章围绕这个目标讲 DDPG、TD3、SAC：它们都是连续动作下的 off-policy actor-critic，只是稳定性和探索处理不同。

连续动作空间的挑战

DQN 需要计算：

$$\max_a Q(s', a)$$

如果动作空间是离散且规模不大，可以枚举所有动作。
但连续动作空间无法枚举，例如机器人控制中的力矩、速度、角度。

因此需要直接学习一个 actor 输出动作：

$$a = \mu_{\theta}(s)$$

或者输出动作分布：

$$a \sim \pi_{\theta}(\cdot | s)$$

从 DQN 到 Actor 的推理过程

DQN 在离散动作中可以做：

$$a^* = \operatorname{argmax}_a Q(s, a)$$

连续动作下，这个 maximization 本身就是一个困难优化问题。如果每次决策都在动作空间里搜索最大 Q，代价很高，也不稳定。

DDPG/TD3/SAC 的思路是额外训练一个 actor，让 actor 学会直接输出“critic 认为高价值”的动作：

```
critic: 评价 Q(s, a)
actor: 产生让 Q(s, a) 尽量大的动作
```

所以连续控制里的 actor 不是装饰，它是在替代 DQN 中对动作的显式枚举或 argmax。

DDPG

DDPG 是 Deep Deterministic Policy Gradient。

它适用于连续动作空间，是 off-policy actor-critic。

组件：

- Actor：确定性策略 $\mu_\theta(s)$ 。
- Critic：动作价值函数 $Q_w(s,a)$ 。
- Replay buffer。
- Target actor。
- Target critic。

Critic target：

$$y = r + \gamma Q_w(s', \mu_\theta(s'))$$

Critic loss：

$$L(w) = \mathbb{E}[(Q_w(s,a) - y)^2]$$

Actor objective：

$$J(\theta) = \mathbb{E}[Q_w(s, \mu_\theta(s))]$$

Actor 更新：

$$\begin{aligned} & \nabla_\theta J \\ &= \mathbb{E}[\nabla_a Q_w(s,a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)] \end{aligned}$$

这个梯度来自 chain rule。目标是让：

$$J(\theta) = Q_w(s, \mu_\theta(s))$$

变大。参数 θ 先影响 actor 输出的动作 $\mu_\theta(s)$ ，动作再影响 critic 给出的 $Q_w(s,a)$ 。所以更新方向分两段：

- $\nabla_a Q_w(s,a)$ ：critic 告诉 actor，动作往哪个方向改能提高 Q。
- $\nabla_\theta \mu_\theta(s)$ ：actor 告诉参数，怎么改参数能让动作往那个方向移动。

面试表达：

DDPG 用 critic 对连续动作求梯度，再通过 actor 反传到策略参数，相当于用 actor 学一个可微的 argmax。

探索：

DDPG 的策略是 deterministic，因此训练时通常给动作加噪声：

$$a_t = \mu_\theta(s_t) + \varepsilon_t$$

DDPG 的问题

DDPG 比较脆弱，常见问题：

- Q 值过估计。
- 对超参数敏感。
- exploration noise 难调。
- actor 可能利用 critic 的错误高估。
- 训练容易不稳定。

TD3 就是针对 DDPG 的几个问题做改进。

TD3

TD3 是 Twin Delayed DDPG。

核心改进：

1. Clipped Double Q-learning。
2. Delayed policy update。
3. Target policy smoothing。

Clipped Double Q-learning

TD3 学两个 critic：

$$Q_1(s, a), Q_2(s, a)$$

target 使用两者较小值：

$$y = r + \gamma \min_i Q_i^-(s', a')$$

这样可以缓解 overestimation bias。

Delayed Policy Update

Critic 更新多次后，actor 再更新一次。

直觉：

critic 估计更可靠后，再用它指导 actor，减少 actor 被错误 Q 值带偏。

Target Policy Smoothing

target action 加入 clipped noise：

$$a' = \mu^-(s') + \varepsilon_{clip}$$

再用 a' 计算 target。

直觉：

让 critic 不要对动作空间中的尖锐错误峰值过拟合，使 Q 函数更平滑。

TD3 三个改进的共同逻辑

TD3 的三个 trick 都围绕同一个问题：actor 很容易利用 critic 的错误。

改进	解决的问题	推理
双 Q 取最小	Q 过估计	如果一个 critic 偶然高估，取 min 可以更保守。
延迟 actor 更新	actor 跟着错误 critic 跑	先多训几步 critic，让评价器更靠谱，再更新 actor。
target smoothing	critic 对尖锐动作峰值过拟合	target 动作附近加噪声，让 Q 在局部更平滑。

一句话：

TD3 不是三个孤立技巧，而是在防止 deterministic actor 钻 critic 估计误差的空子。

SAC

SAC 是 Soft Actor-Critic，是现代连续控制中非常常用的算法。

核心思想：

最大化 reward 的同时最大化策略 entropy。

目标：

$$\mathbb{E}[\sum_t \gamma^t (r_t + \alpha H(\pi(\cdot|s_t)))]$$

或者写成：

$$\mathbb{E}[\sum_t \gamma^t (r_t - \alpha \log \pi(a_t|s_t))]$$

其中 α 是 temperature，控制 entropy 权重。

SAC 的组件

SAC 通常包含：

- Stochastic actor： $\pi_\theta(a|s)$ 。
- 两个 Q network： Q_1, Q_2 。
- target Q networks。

- replay buffer。
- temperature parameter α 。

Soft Q target:

$$y = r + \gamma [\min_i Q_i^-(s', a') - \alpha \log \pi(a'|s')]$$

其中:

$$a' \sim \pi_\theta(.|s')$$

Actor objective:

$$J_\pi(\theta) = \mathbb{E}[\alpha \log \pi_\theta(a|s) - \min_i Q_i(s,a)]$$

最小化这个 loss 等价于选择高 Q 且高 entropy 的动作分布。

SAC Soft Target 的推理过程

普通 Bellman target 是:

$$y=r+\gamma Q(s',a')$$

SAC 的目标不是只最大化 reward, 而是最大化 reward 加 entropy。对某个动作来说, entropy 项可以写成:

$$-\alpha \log \pi(a'|s')$$

所以 soft value 可以理解为:

$$Q(s',a')-\alpha \log \pi(a'|s')$$

代回 Bellman target, 就得到:

$$y=r+\gamma[\min_i Q_i^-(s',a')-\alpha \log \pi(a'|s')]$$

为什么 actor loss 是:

$$J_\pi(\theta)=\mathbb{E}[\alpha \log \pi_\theta(a|s)-\min_i Q_i(s,a)]$$

因为实现里通常最小化 loss。最小化这个式子等价于:

- 最大化 Q: 让动作回报高。
- 最大化 entropy: 因为最小化 $\log \pi$ 倾向避免策略过度确定。

面试里可以说:

SAC 把探索写进目标函数，而不是只靠外加噪声。Soft target 中的 $-\alpha \log \pi$ 会奖励更高熵的动作分布，因此策略既追求高 Q，也保持足够随机性。

Entropy Regularization 的意义

SAC 不只是把 entropy 当作训练 trick，而是把它放进目标函数。

好处：

- 鼓励探索。
- 避免策略过早确定。
- 对多模态最优策略更友好。
- 训练通常比 DDPG/TD3 更稳定。

α 控制探索程度：

- α 大：更重视 entropy，策略更随机。
- α 小：更重视 reward，策略更确定。

现代 SAC 常自动调节 α ，让策略 entropy 接近 target entropy。

DDPG vs TD3 vs SAC

维度	DDPG	TD3	SAC
策略	deterministic	deterministic	stochastic
数据	off-policy	off-policy	off-policy
critic	单 Q	双 Q	双 Q
探索	外加噪声	外加噪声	entropy 驱动
稳定性	较弱	比 DDPG 强	通常更强
适用	连续控制	连续控制	连续控制、鲁棒训练

本章例子：连续控制怎么选算法

例子 1：机械臂控制

机械臂每个关节输出连续力矩：

```
a = [torque_shoulder, torque_elbow, torque_wrist]
```

DQN 不适合，因为动作不是离散的。DDPG、TD3、SAC 可以直接输出连续动作。

例子 2：DDPG 的问题

如果 critic 对某个连续动作附近估值过高，actor 会被 critic 引导到这个错误高值区域。

这就是 DDPG 容易不稳定的原因：

critic 估错 → actor 追错 → 数据分布变差 → critic 更难学准

例子 3：TD3 怎么修

TD3 用两个 critic：

$$\min(Q_1, Q_2)$$

如果一个 critic 过高估计，另一个没那么高，取 min 可以降低过估计传播。

例子 4：SAC 为什么适合探索

如果机器人需要学会走路，早期策略不能太确定，否则很容易卡在局部动作模式。SAC 的 entropy 项鼓励策略保持随机性：

既要拿高 reward，也要保持足够探索

面试表达：

DDPG、TD3、SAC 都处理连续动作。DDPG 是基础 actor-critic，TD3 主要修过估计和不稳定，SAC 用最大熵目标把探索写进优化目标。

本章面试高频问题

1. 为什么 DQN 不适合连续动作？

DQN 需要对动作枚举并取 $\max_a Q(s, a)$ 。连续动作空间无法枚举，直接求 argmax 也通常很难，因此需要 actor 直接输出连续动作。

2. DDPG 的核心是什么？

DDPG 是连续动作空间下的 off-policy actor-critic。Actor 输出确定性动作，critic 评估 $Q(s, a)$ ，actor 通过最大化 critic 给出的 Q 值来更新。

3. TD3 相比 DDPG 做了哪些改进？

TD3 使用双 critic 取最小值缓解过估计，延迟 actor 更新提高稳定性，并使用 target policy smoothing 平滑 Q 函数。

4. SAC 为什么稳定？

SAC 使用最大熵目标，鼓励高 reward 和高 entropy 的策略；同时使用双 Q 网络、target network 和 replay buffer。Entropy regularization 带来更好的探索和鲁棒性。

5. SAC 中 α 是什么？

α 是 entropy temperature, 控制 reward 和 entropy 的权衡。 α 越大, 策略越随机、探索越强; α 越小, 策略越偏向最大化 reward。

本章练习

1. 写出 DDPG critic target 和 actor objective。
2. 解释 TD3 的三个 trick。
3. 写出 SAC soft Q target。
4. 对比 deterministic policy 和 stochastic policy。

第 11 章：Model-based RL（Model-based Reinforcement Learning，基于模型的强化学习）、Offline RL（离线强化学习）、Imitation Learning 与 RLHF（Reinforcement Learning from Human Feedback，基于人类反馈的强化学习）

本章符号说明

符号/缩写	English	中文	含义
RL	Reinforcement Learning	强化学习	通过交互或数据学习策略以最大化长期 reward。
Offline RL	Offline Reinforcement Learning	离线强化学习	只用固定数据集学习策略，不继续和环境交互。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好训练 reward model，再优化 policy。
$P(s' s, a)$	Transition Probability	状态转移概率	环境 dynamics 的概率形式。
$R(s, a)$	Reward Function	奖励函数	环境或 reward model 给出的奖励。
D	Dataset	数据集	offline RL 或 imitation learning 使用的固定样本集合。
$f_{\phi}(s, a)$	Dynamics Model	动力学模型	参数 ϕ 下预测下一状态的模型。
BC	Behavior Cloning	行为克隆	把专家示范当监督学习来模仿。
IRL	Inverse Reinforcement Learning	逆强化学习	从专家行为反推出 reward function。
SFT	Supervised Fine-Tuning	监督微调	RLHF 中用人工示范数据微调语言模型。
RM	Reward Model	奖励模型	根据偏好数据预测 response 质量的模型。
DPO	Direct Preference Optimization	直接偏好优化	不显式跑 PPO 的偏好优化方法。
KL	Kullback-Leibler Divergence	KL 散度	RLHF 中常用来限制 policy 偏离 reference model。

符号/缩写	English	中文	含义
DQN / PPO / SAC	Deep Q-Network / Proximal Policy Optimization / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 软 Actor-Critic	文中作为 model-free 或 RLHF 优化阶段的代表算法。
BCQ	Batch-Constrained deep Q-learning	批约束深度 Q 学习	offline RL 代表算法之一。
BEAR	Bootstrapping Error Accumulation Reduction	bootstrap 误差累积降低	offline RL 代表算法之一。
CQL	Conservative Q-Learning	保守 Q 学习	通过保守估计 Q 值处理 OOD action 的 offline RL 方法。
IQL	Implicit Q-Learning	隐式 Q 学习	不显式查询 OOD action 的 offline RL 方法。
OOD	Out-of-Distribution	分布外	数据集中缺少覆盖的状态或动作。

本章目标

本章补充面试中越来越常见的进阶主题。你需要掌握：

- model-free vs model-based RL。
- offline RL 的核心难点。
- imitation learning 和 behavior cloning。
- inverse reinforcement learning。
- RLHF 的基本流程。

本章主线

前 10 章主要讨论在线 model-free 学习。本章换成四类现实约束：能不能学或使用环境模型，能不能继续交互，能不能从专家示范学，能不能用人类偏好定义 reward。Model-based RL、Offline RL、Imitation Learning、RLHF 分别对应这些约束，是面试里从经典 RL 扩展到真实系统和大模型对齐的关键桥梁。

Model-free vs Model-based

Model-free RL 不显式学习环境模型，直接学习 value 或 policy：

- Q-learning。
- DQN。
- PPO。
- SAC。

Model-based RL 显式学习或使用环境模型：

$$P(s' | s, a), R(s, a)$$

然后基于模型进行 planning 或生成模拟数据。

Model-based RL 的流程

典型流程：

1. 收集环境交互数据。
2. 学习 dynamics model：

$$s_{t+1} \approx f_p^{hi}(s_t, a_t)$$

3. 用模型进行 planning 或 rollout。
4. 根据真实环境反馈继续修正模型。

优点：

- sample efficiency 可能更高。
- 可以通过模型进行规划。
- 在真实交互昂贵的场景中有吸引力。

缺点：

- 模型误差会累积。
- 长 horizon rollout 容易偏离真实环境。
- 高维复杂环境中 dynamics 很难学准。

面试表达：

Model-based RL 的核心优势是样本效率，但主要问题是 model bias。学到的 dynamics model 有误差，规划或 rollout 会放大这些误差，尤其在长时序任务中更明显。

Offline RL

Offline RL，也叫 batch RL，目标是在不继续和环境交互的情况下，只用固定数据集学习策略。

数据集：

$$D = \{(s, a, r, s')\}$$

这些数据来自某个 behavior policy，可能是人类、旧系统或混合策略。

与 online RL 的区别：

- online RL 可以探索并收集新数据。
- offline RL 只能使用已有数据。

Offline RL 的难点

核心问题：distribution shift。

学习到的新策略可能选择数据集中很少出现甚至没出现过的动作。此时 Q 函数对这些 out-of-distribution actions 的估计不可靠。

如果 Q 函数错误高估了某些 OOD 动作，policy improvement 会进一步偏向这些动作，导致性能崩溃。

这也叫 extrapolation error。

面试表达：

Offline RL 难在不能通过真实环境纠错。策略可能选择数据分布之外的动作，而 critic 对这些动作的 Q 值没有可靠监督，容易产生过估计并被 policy 利用。

Offline RL 的常见思路

常见方法会让策略不要偏离数据分布太远，或让 Q 值对 OOD 动作更保守。

几类思路：

- Behavior regularization：限制 learned policy 接近 behavior policy。
- Conservative value estimation：压低 OOD action 的 Q 值。
- Importance weighting：修正策略分布差异。
- Sequence modeling：把决策问题转成条件序列建模，例如 Decision Transformer。

代表算法了解即可：

- BCQ。
- BEAR。
- CQL。
- IQL。
- Decision Transformer。

Imitation Learning

Imitation Learning 从专家数据中学习策略。

专家数据：

$$(s, a_{expert})$$

目标：让 agent 模仿专家动作。

Behavior Cloning

Behavior Cloning，简称 BC，是最简单的 imitation learning。

把专家数据当监督学习：

$$\max_{\theta} \log \pi_{\theta}(a_{expert} | s)$$

或者最小化：

$$L(\theta) = -\log \pi_{\theta}(a_{\text{expert}} | s)$$

优点：

- 简单。
- 稳定。
- 不需要 reward。
- 训练像监督学习。

缺点：

- distribution shift。
- compounding error。
- 遇到专家数据中没覆盖的状态时容易出错。

为什么会 compounding error?

训练时模型只见过专家状态分布。部署时一旦模型犯小错，进入专家没覆盖的状态，就更容易继续犯错，误差逐步累积。

DAgger

DAgger 是 Dataset Aggregation。

核心思路：

1. 用当前策略和环境交互。
2. 让专家给当前策略访问到的状态标注正确动作。
3. 把新数据加入数据集。
4. 重新训练策略。

DAgger 通过收集 learner 自己会遇到的状态，缓解 behavior cloning 的分布偏移。

Inverse Reinforcement Learning

IRL 的目标不是直接模仿动作，而是从专家行为中反推 reward function。

直觉：

专家之所以这么行动，是因为它在优化某个隐藏 reward。IRL 希望恢复这个 reward，然后再用 RL 学策略。

适合场景：

- 专家动作可观察。
- reward 很难手工设计。
- 希望学到可迁移的偏好或目标。

缺点：

- 问题不适定：多个 reward 都可能解释同一行为。

- 计算复杂。
- 实践中常被更直接的 imitation 或 preference learning 替代。

RLHF

RLHF 是 Reinforcement Learning from Human Feedback，常用于大语言模型对齐。

典型流程：

1. Pretraining：在大规模语料上预训练基础模型。
2. SFT：用人工示范数据做 supervised fine-tuning。
3. Reward Model：收集人类偏好比较，训练 reward model。
4. RL Optimization：用 PPO 等算法优化 policy，使其获得更高 reward model 分数。

偏好数据形式：

$$(prompt, response_{chosen}, response_{rejected})$$

Reward model 学习：

$$r_{chosen} > r_{rejected}$$

常见 pairwise loss：

$$L = -\log \sigma(r_{chosen} - r_{rejected})$$

RLHF 中的 PPO

在 RLHF 中，policy 是语言模型。动作是 token，trajectory 是生成序列。

优化目标通常包含：

- reward model 分数。
- KL penalty，限制新模型不要偏离 reference model 太远。

形式：

$$r_{total} = r_{RM} - \beta KL(\pi \parallel \pi_{ref})$$

为什么需要 KL penalty？

- 防止模型为了骗 reward model 而偏离语言质量。
- 保持和 SFT/reference model 的行为接近。
- 提高训练稳定性。

RLHF 的常见问题

- Reward hacking：模型找到 reward model 漏洞。

- Preference data 噪声。
- Reward model 泛化问题。
- PPO 训练不稳定。
- 对齐目标难以完全由标注偏好表达。

近年也常见不用 RL 的偏好优化方法，例如 DPO。面试中可以简单提到：

DPO 直接从偏好数据推导监督式目标，避免显式训练 reward model 后再跑 PPO，工程上更简单。

本章例子：四类高级 RL 场景

例子 1：Model-based RL 做机器人规划

机器人先学习一个 dynamics model：

$$\hat{P}(s'|s,a)$$

然后在模型里模拟不同动作序列：

动作序列 1 -> 预测会撞墙
动作序列 2 -> 预测能到目标点
动作序列 3 -> 预测路径更短但风险高

再选择预测回报最高的动作序列执行。

例子 2：Offline RL 做历史推荐日志

公司已经有大量推荐日志：

用户状态、展示内容、点击/停留/购买

Offline RL 希望只用这些历史数据学习新策略。但难点是新策略可能推荐历史日志里很少出现的内容，价值估计容易外推错误。

所以 offline RL 常需要保守约束：不要让策略偏离数据分布太远。

例子 3：Imitation Learning 做驾驶

Behavior Cloning 可以用人类驾驶数据训练：

输入：摄像头、速度、导航
输出：方向盘角度、油门、刹车

问题是模型一旦偏离人类数据分布，就可能进入训练集中没见过的状态。DAgger 的思路是让专家在模型实际访问到的状态上继续标注，补齐分布偏移。

例子 4：RLHF 做聊天模型

RLHF pipeline:

1. 用 SFT 让模型学会基本回答格式。
2. 收集同一个 prompt 下多个回答的人类偏好。
3. 训练 reward model。
4. 用 PPO 等方法优化策略，同时用 KL 限制模型偏离参考模型。

面试表达:

Model-based、Offline、Imitation、RLHF 的共同点是都在解决“真实交互成本高或风险大”的问题：要么学模型模拟，要么复用历史数据，要么模仿专家，要么用人类偏好构造 reward。

本章面试高频问题

1. Model-free 和 model-based RL 区别？

Model-free 不显式学习环境模型，直接学习 policy 或 value。Model-based 学习或使用环境 dynamics 和 reward model，并基于模型 planning 或生成数据。

2. Model-based RL 的主要问题是什么？

模型误差。学到的 dynamics model 不可能完全准确，长 horizon planning 会累积误差，导致策略利用模型漏洞而在真实环境中表现差。

3. Offline RL 为什么难？

因为只能使用固定数据集，不能通过环境探索纠错。策略可能选择数据分布之外的动作，而 Q 函数对这些动作估计不可靠，容易产生 extrapolation error。

4. Behavior Cloning 的问题？

BC 把模仿学习当监督学习，简单稳定，但有 distribution shift 和 compounding error。模型部署后遇到专家数据没覆盖的状态，错误会逐步累积。

5. RLHF 的基本流程？

先预训练语言模型，再用示范数据 SFT，然后用人类偏好训练 reward model，最后用 PPO 等 RL 算法在 KL 约束下优化模型输出。

本章练习

1. 用一句话区分 online RL 和 offline RL。
2. 解释 offline RL 中的 extrapolation error。
3. 对比 Behavior Cloning 和 RL。
4. 画出 RLHF 的四阶段流程。

第 12 章：强化学习面试速查与项目准备

本章符号说明

本章是速查表，会集中复现前面章节的核心符号：

符号/缩写	English	中文	含义
MDP	Markov Decision Process	马尔可夫决策过程	RL 标准建模。
$S / A / R$	State / Action / Reward	状态 / 动作 / 奖励	MDP 三个基础元素。
G_t	Return	回报 / 累计折扣奖励	从时刻 t 开始的累计收益。
$V_{\pi}(s)$	State-value Function	状态价值函数	策略 π 下状态 s 的长期价值。
$Q_{\pi}(s,a)$	Action-value Function	动作价值函数	策略 π 下状态-动作对 (s,a) 的长期价值。
$A_{\pi}(s,a)$	Advantage Function	优势函数	动作相对状态平均价值的优势；这里的 A 不是 action space。
$J(\theta)$	Objective Function	目标函数	policy optimization 中要最大化的目标。
$L(\theta)$	Loss Function	损失函数	神经网络训练中要最小化的目标。
KL	Kullback-Leibler Divergence	KL 散度	衡量两个策略分布的差异。
DQN / PPO / SAC	Deep Q-Network / Proximal Policy Optimization / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 软 Actor-Critic	面试中最高频的三类 deep RL 算法。
RL / RLHF	Reinforcement Learning / Reinforcement Learning from Human Feedback	强化学习 / 基于人类反馈的强化学习	本套课程和对齐流程中的核心概念。
DP / MC / TD	Dynamic Programming / Monte Carlo / Temporal Difference	动态规划 / 蒙特卡洛 / 时序差分	价值估计和控制的基础方法。
SARSA	State-Action-Reward-State-Action	状态-动作-奖励-状态-动作	on-policy TD control 算法。
REINFORCE	REINFORCE algorithm	REINFORCE 算法	Monte Carlo policy gradient 基线算法。

符号/缩写	English	中文	含义
A2C / A3C	Advantage Actor-Critic / Asynchronous Advantage Actor-Critic	优势 Actor-Critic / 异步优势 Actor-Critic	actor-critic 的同步和异步版本。
GAE	Generalized Advantage Estimation	广义优势估计	PPO 常用的 advantage 估计方法。
DDPG / TD3	Deep Deterministic Policy Gradient / Twin Delayed DDPG	深度确定性策略梯度 / 双延迟 DDPG	连续动作控制算法。
UCB	Upper Confidence Bound	置信上界	bandit 中基于不确定性 bonus 的探索方法。
MLP	Multi-Layer Perceptron	多层感知机	CartPole DQN 项目中常用的小型神经网络。
PG	Policy Gradient	策略梯度	直接优化策略参数的一类方法。

本章目标

本章用于面试前快速复习。你需要整理：

- 核心概念速查。
- 常见算法横向对比。
- 高频公式。
- 高频问答。
- 项目表达模板。

本章主线

本章不是引入新算法，而是把前面所有内容压缩成面试表达结构。复习顺序是：先用一张图串起 MDP、Bellman、DP、MC/TD、DQN、Policy Gradient、Actor-Critic、PPO/SAC；再横向比较算法；最后把项目按“问题建模、状态动作奖励、算法选择、训练稳定性、指标和失败案例”讲清楚。

核心概念速查

MDP：

$$(S, A, P, R, \gamma)$$

Return：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Value：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

Q value:

$$Q_{\pi}(s,a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

Advantage:

$$A_{\pi}(s,a) = Q_{\pi}(s,a) - V_{\pi}(s)$$

Bellman expectation:

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma V_{\pi}(S_{t+1}) | S_t=s]$$

Bellman optimality:

$$Q^*(s,a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') | S_t=s, A_t=a]$$

TD error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Policy Gradient:

$$\nabla J(\theta) = \mathbb{E}[\nabla \log \pi_{\theta}(a|s) A(s,a)]$$

PPO ratio:

$$r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{old}(a_t|s_t)$$

PPO clipped objective:

$$\mathbb{E}[\min(r_t A_p, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_p)]$$

SAC soft target:

$$y = r + \gamma [\min_i Q_i^-(s', a') - \alpha \log \pi(a' | s')]$$

算法横向对比

算法	类型	on/off-policy	关键点	适用
Policy Iteration	DP	模型已知	evaluation + improvement	小规模 MDP

算法	类型	on/off-policy	关键点	适用
Value Iteration	DP	模型已知	Bellman optimality backup	小规模 MDP
Monte Carlo	value estimation	on-policy 常见	完整 return	episodic task
TD(0)	value estimation	on-policy 常见	bootstrap	在线估值
SARSA	value-based	on-policy	target 用实际 a'	表格控制
Q-learning	value-based	off-policy	target 用 max	表格控制
DQN	value-based	off-policy	replay + target network	离散动作
REINFORCE	policy-based	on-policy	Monte Carlo PG	教学基础
A2C/A3C	actor-critic	on-policy	critic 降方差	离散/连续
PPO	actor-critic	on-policy	clipped objective	通用稳定基线
DDPG	actor-critic	off-policy	deterministic actor	连续动作
TD3	actor-critic	off-policy	double Q + delayed update	连续动作
SAC	actor-critic	off-policy	maximum entropy	连续动作

高频概念问答

1. RL 为什么难？

因为 RL 同时面对 delayed reward、探索与利用、非 i.i.d. 数据、动作影响未来数据分布、credit assignment 和训练稳定性问题。

2. Bellman 方程为什么重要？

Bellman 方程描述了价值函数的递归自洽关系：当前价值等于即时 reward 加未来价值。绝大多数 value-based 和 actor-critic 方法都直接或间接基于 Bellman backup。

3. On-policy 和 off-policy 区别？

On-policy 学习当前采样策略的价值或改进当前策略。Off-policy 用 behavior policy 的数据学习另一个 target policy，样本效率更高，但有分布偏移风险。

4. MC 和 TD 区别？

MC 用完整 episode 的真实 return，不 bootstrap，bias 小但 variance 大。TD 用一步 reward 加下一状态价值估计，能在线更新，variance 小但有 bootstrap bias。

5. SARSA 和 Q-learning 区别？

SARSA 是 on-policy，target 使用实际采样的下一个动作。Q-learning 是 off-policy，target 使用下一状态最大 Q 值。

6. DQN 的两个核心稳定技巧？

Experience replay 和 target network。前者打破样本相关性并复用样本，后者稳定 bootstrap target。

7. Double DQN 解决什么？

缓解 DQN 中 $\max$ 操作造成的 Q 值过估计。它用 online network 选动作，用 target network 评估动作。

8. Policy Gradient 怎么理解？

如果某个动作带来正 advantage，就增加该动作在该状态下的概率；如果 advantage 为负，就降低概率。公式是：

$$\nabla J = \mathbb{E}[\nabla \log \pi(a|s) A(s,a)]$$

9. Baseline 为什么不引入 bias？

因为 baseline 不依赖 action，而：

$$\mathbb{E}_{a \sim \pi}[\nabla \log \pi(a|s)] = 0$$

因此减去 baseline 只改变方差，不改变梯度期望。

10. PPO 为什么常用？

PPO 用 clipped objective 限制新旧策略差异，稳定性好、实现简单、适用面广，是 on-policy actor-critic 中非常常用的工程基线。

11. SAC 为什么适合连续控制？

SAC 是 off-policy，样本效率较高；最大熵目标鼓励探索并提升稳定性；双 Q 网络缓解过估计，因此在连续控制任务中表现稳定。

12. Offline RL 为什么难？

因为不能继续探索收集数据。学习策略可能选择数据集中没覆盖的动作，critic 对这些动作估计不可靠，容易出现 extrapolation error。

面试中算法讲解模板

回答某个算法时，按这个顺序讲：

1. 它解决什么问题？
2. 是 value-based、policy-based 还是 actor-critic？
3. 是 on-policy 还是 off-policy？
4. 核心目标函数或更新公式是什么？
5. 关键 trick 是什么？
6. 相比前一个算法改进在哪里？
7. 局限是什么？

例如讲 DQN：

DQN 是把 Q-learning 扩展到高维状态输入的 deep RL 算法。它用神经网络近似 $Q(s,a)$ ，适合离散动作空间。核心 target 是 $r + \gamma \max_{a'} Q_{\text{target}}(s',a')$ 。为了稳定训练，它使用 replay buffer 打破样本相关性、复用历史经验，并使用 target network 稳定 Bellman target。它的局限是难以处理连续动作空间，并且存在 Q 值过估计问题，后续 Double DQN 对此做了改进。

项目准备建议

面试前建议至少准备一个可讲清楚的 RL 小项目。

优先级：

- 1. CartPole with DQN。
- 2. LunarLander with PPO。
- 3. Pendulum or HalfCheetah with SAC。
- 4. 简化推荐系统 bandit。
- 5. RLHF toy example，例如偏好数据训练 reward model。

面试例子索引

准备面试时，建议把算法和例子绑定记忆：

算法/概念	推荐例子	为什么适合
MDP / Bellman	GridWorld	状态、动作、转移、奖励最直观。
DP	已知地图的 GridWorld	可以明确说明“已知环境模型”。
MC	Blackjack	episode 短，完整 return 容易得到。
TD	在线推荐或持续控制	不想等 episode 结束，每步都能更新。
SARSA / Q-learning	Cliff Walking	on-policy 与 off-policy 差异明显。
Bandit	广告推荐冷启动	探索和利用最直接。
Function Approximation	Atari 或推荐系统	状态空间无法枚举。
DQN	CartPole / Atari	离散动作 + Q 网络。
Policy Gradient	连续控制机器人	动作连续，难以枚举 max。
Actor-Critic / GAE	游戏或机器人控制	Actor 选动作，Critic 降低方差。
PPO	RLHF 或 LunarLander	策略更新稳定，工程常用。
DDPG / TD3 / SAC	机械臂、MuJoCo	连续动作控制。
Offline RL	历史推荐日志	只能用离线数据，不能随便探索。
Imitation Learning	自动驾驶示范数据	从专家轨迹学习行为。
RLHF	聊天模型对齐	用人类偏好构造 reward。

回答时不要只说“我用 DQN 做 CartPole”。更好的结构是：

CartPole 是离散动作控制任务，状态低维连续，动作是 left/right，reward 是存活步数。DQN 用 Q 网络输出两个动作价值，配合 replay buffer 和 target network 稳定训练。

项目表达模板

项目背景：
任务建模：
状态空间：
动作空间：
奖励设计：
算法选择：
训练流程：
关键工程细节：
实验指标：
遇到的问题：
改进方向：

CartPole DQN 项目讲法

任务建模：

- State：小车位置、速度、杆角度、角速度。
- Action：向左或向右施加力。
- Reward：每保持一步平衡得到 1。
- Done：杆倾斜过大或小车越界。

算法选择：

- 动作空间离散，适合 DQN。
- 用 MLP 近似 Q 函数。
- epsilon-greedy 探索。
- replay buffer 存储 transition。
- target network 稳定 target。

可以强调的问题：

- epsilon decay 如何设置。
- replay buffer 大小。
- target network 更新频率。
- loss 曲线不稳定是否正常。
- reward 是否能达到环境上限。

LunarLander PPO 项目讲法

任务建模：

- State：位置、速度、角度、角速度、腿部接触信息。
- Action：离散喷气控制。
- Reward：接近着陆点、平稳降落、节省燃料等。

算法选择：

- PPO 稳定且适合 policy optimization。
- 使用 GAE 估计 advantage。
- 使用 entropy bonus 保持探索。
- 使用 clipped objective 限制策略更新。

可以强调：

- PPO 是 on-policy，采样效率不如 DQN/SAC。
- 但训练稳定，调参相对友好。
- advantage normalization 对训练很重要。

推荐系统 Bandit 项目讲法

任务建模：

- State/context：用户画像、上下文、历史行为。
- Action：推荐某个 item 或策略。
- Reward：点击、停留、转化、长期留存。

算法选择：

- 如果动作不影响长期状态，可先用 contextual bandit。
- 可比较 epsilon-greedy、UCB、Thompson Sampling。

注意点：

- 线上探索有风险，需要控制流量。
- reward 延迟和偏差要处理。
- 离线评估有 selection bias。

面试前最终检查清单

- 能手写 Bellman expectation 和 optimality equation。
- 能解释 MC、TD、DP 的区别。
- 能写出 SARSA 和 Q-learning 更新公式。
- 能解释 DQN 两个核心 trick。
- 能解释 Double DQN 为什么缓解过估计。
- 能推导 policy gradient 的 log-derivative trick。
- 能证明 baseline 不引入 bias。
- 能解释 Actor-Critic 和 GAE。

- 能手写 PPO clipped objective。
- 能解释 SAC 的最大熵目标。
- 能说明 offline RL 的 extrapolation error。
- 能讲一个完整 RL 项目。

30 分钟口述复习路线

1. 5 分钟：MDP、return、V/Q、Bellman。
2. 5 分钟：DP、MC、TD、SARSA、Q-learning。
3. 5 分钟：DQN、Double DQN、Dueling、Replay。
4. 5 分钟：Policy Gradient、baseline、Actor-Critic、GAE。
5. 5 分钟：PPO、DDPG、TD3、SAC。
6. 5 分钟：offline RL、RLHF、项目总结。